MedGate-FL: Capability-Isolated Federated Adapters for Tiered Medical AI Access

[Preliminary technical report — synthetic validation only, real-data tier license-gated]

VectorSophie

Draft of August 26, 2026 — Phases 0–5 complete on two synthetic fixtures (null-signal and hierarchical); real-data tier license-gated

Abstract

Federated learning is often described as protecting patient privacy because raw data never leaves an institution, but this protects data *locality*, not the gradients, representations, or final model federation produces. We study *capability isolation*: whether a federated model can expose a public, coarse-grained diagnostic capability while keeping a fine-grained capability restricted to authorized users, and whether that restriction survives realistic attacks. We implement MedGate-FL under six training objectives, evaluated against a fair pretrained-baseline family, four attacker models, a precisely-named attack suite including a genuine parameter-space adapter-recovery attack, a pairwise-masking privacy layer, DP-SGD, and five unlearning methods. FLamby Fed-ISIC2019 remains license-gated, so results here run on two synthetic fixtures instead: a null-signal smoke-test fixture and a new hierarchical fixture, independently verified learnable, built from Fed-ISIC2019's own class-imbalance statistics. On the hierarchical fixture, fair baselines show a real coarse/fine capability gap, but capability isolation does not show a clean win over it: at 5 seeds, the best validation-selected probe recovers as much or more fine-grained information than authorized users get, for every isolation objective tested – an open, negative finding, reported rather than suppressed. This repair pass also fixed three validity bugs a review found: an adapter-recovery attack that completed a matrix whose rank truncation was a no-op; a masking module claiming an information-theoretic concealment guarantee its Gaussian mask does not have; and a DP budget that reported one federated round's epsilon as the whole training run's. Whether capability isolation works on real medical images remains unknown.

1 Introduction

Cross-silo federated learning lets multiple hospitals train a shared model without centralizing patient data [McMahan et al., 2017]. This is frequently summarized as "federated learning protects privacy," but the protection is narrower than that: only the raw training images stay local. The gradients or weight updates each institution sends are still exposed to the aggregating party [Zhu et al., 2019]; the final trained model, once distributed, is exposed to whoever receives it; and nothing about the FedAvg protocol itself limits what that model can be made to reveal under later probing, fine-tuning, or querying. Prior work on medical federated learning *[background source, not an empirical claim of this paper – Ogier du Terrail et al. [2022]]* and on securing federated training generally treats three problems separately: protecting the training updates (secure aggregation [Bonawitz et al., 2017], differential privacy [Abadi et al., 2016]), protecting model intellectual property after distribution, and authorizing who may query or fine-tune a deployed model.

This paper studies a fourth, distinct problem that sits underneath all three: a single federated model may need to expose a coarse, low-risk capability to a broad audience while keeping a fine-grained, higher-risk diagnostic capability restricted to an authorized tier. A concrete instance, and the one this paper builds around, is skin-lesion classification: a coarse melanocytic/keratinocytic/other signal might be safe to expose broadly, while the eight-way fine diagnosis (including melanoma vs. benign nevus) is exactly the output a clinical safety or liability framework would want gated. We call keeping that fine capability restricted, while a coarse capability remains useful, **capability isolation**.

The naive approach – train both heads, only expose the coarse one – is not capability isolation, because the fine information may still be linearly or near-linearly recoverable from the shared backbone's representation even though the fine head itself is hidden (this is exactly the `hidden_fine_head` arm we implement and probe in §9). Establishing whether that residual leakage is small in practice, under realistic attacker budgets, is the empirical core of this paper: it is not assumed, it is measured, with a battery of representation probes (linear, nonlinear, k-NN, few-shot) and reconstruction/extraction attacks (§8).

Contributions. (1) A capability-isolation training objective and architecture (§5) compared against five simpler alternatives under matched compute, not assumed superior. (2) An attacker-explicit evaluation across four threat models (§4) with attacker knowledge, access, and budget recorded for every attack result. (3) A conservative cryptographic/authorization layer (AES-GCM, centralized IAM, never RSA-encrypting tensors directly) and a from-scratch secure-aggregation simulation, with an honestly reported instance of building the wrong test for a security property and correcting it (§7.3). (4) A revocation-vs-unlearning study that operationalizes, rather than asserts, the project's central non-claim that revoking a key does not remove a model's already-learned influence (§10). (5) A fully reproducible, open pipeline (§14) in which every number in this draft traces to a config, seed, git commit, and raw JSON file – currently run only on a synthetic fixture, pending a human license-acceptance step for the real data (§6).

2 Background and Motivation

Medical imaging tasks routinely have a natural coarse/fine label hierarchy: a triage-level category (e.g., "possibly malignant, refer") and a diagnosis-level category (e.g., a specific carcinoma subtype). Access to the fine category carries more clinical and liability weight, and in many real deployments would be gated to credentialed users even when a coarse signal is offered as a lower-stakes public or semi-public service (e.g., a consumer-facing triage tool). Federated learning is attractive for training such a model because the fine-grained labels, sourced from multiple hospitals, cannot be centralized – but federating the *training* says nothing about how to gate the *resulting model's* capability once training finishes. That is the gap this paper works in.

3 Related Work

Verification status. Every citation in this section is drawn only from the `VERIFIED` rows of `docs/literature_matrix.csv` (38 of 59 seed-bibliography entries as of this draft); the other 21 are deliberately not cited here even where the project brief's seed bibliography names them, pending a primary-source check (`docs/literature_matrix.md` records exactly which and why). Sections below that would normally cite one of those 21 unverified sources instead name it in the running text and say plainly that it is not yet independently verified, rather than citing it as if it were.

Federated learning. FedAvg [McMahan et al., 2017] established the communication-efficient local-SGD-then-average pattern this paper's baselines and proposed method both build on. FedProx [Li et al., 2020] adds a proximal term for statistical/systems heterogeneity; SCAFFOLD [Karimireddy et al., 2020] corrects client drift with control variates; FedAdam and related adaptive federated optimizers [Reddi et al., 2021] replace plain averaging with a federated adaptive step. Kairouz et al. [2021] survey the broader space. **Gap:** none of these methods, or the survey, address exposing part of a trained model's capability while restricting another part – they optimize *how* a single, uniformly-accessible model is trained, not *what parts of it different recipients are allowed to use*.

Cross-silo medical federated learning and benchmarks. FLamby [Ogier du Terrail et al., 2022] is the direct dataset and baseline source for this paper (§6); it is a benchmark and dataset paper, not a security paper, and makes no capability-isolation claim. Sheller et al. 2020, Rieke et al. 2020, and Teo et al. 2024 make broader medical-FL feasibility/survey claims that would normally belong in this paragraph too, but none of the three is yet independently verified against a primary source (`docs/literature_matrix.csv`), so none is cited here. **Gap:** realistic cross-silo benchmarks like FLamby establish that federated training is feasible on natural institutional splits; they do not evaluate what a distributed model leaks about a restricted sub-task.

Parameter-efficient adaptation and model protection. LoRA [Hu et al., 2022] is the adapter mechanism A_ϕ in MedGate-FL's architecture (§5). LoREnc [Ahn et al., 2026] proposes a training-free spectral-truncation/orthogonal-reparameterization scheme so an unauthorized holder of a LoRA adapter gets structurally degraded outputs while an authorized one recovers full performance – conceptually the same goal as this paper's encrypted-adapter-at-rest design, but LoREnc's mechanism is a transform on the adapter weights themselves (obfuscation), whereas this paper's crypto layer (§7.3) uses conventional AEAD encryption and evaluates the resulting model separately for representation leakage (§9) – the two are complementary, not the same claim, and this paper does not implement or evaluate LoREnc's specific transform. FedShield-LLM (Mia and Amini 2025), an Encryption-Friendly LLM Architecture (Rho et al. 2024), and FL-DABE-BC (Narkedimilli et al. 2024) would also belong in this paragraph, but none is yet independently verified against a primary source, so none is cited. **Gap:** LoRA and LoREnc each solve one piece (parameter efficiency; adapter obfuscation) in isolation from the federated, multi-attacker evaluation this paper runs them through.

Secure aggregation and differential privacy. Practical secure aggregation [Bonawitz et al., 2017] hides individual client updates from the server via pairwise masking over a finite field, with Diffie-Hellman key agreement and Shamir-secret-sharing dropout recovery; this paper's simulated-pairwise-additive-masking module (§7.3) simulates only that mathematical core's cancellation property in a single process, explicitly not the full protocol, and (P0-B, §7.3) uses a Gaussian mask by default rather than the finite-field uniform mask the real protocol's confidentiality argument actually needs – a distinction this paper's earlier drafts blurred and this repair pass corrected. DP-SGD [Abadi et al., 2016] bounds what a trained model's parameters can reveal about any single training example via gradient clipping and noise; this paper applies it per-client via Opacus. Client-level DP [Geyer et al., 2017, McMahan et al., 2018] and the foundational DP definitions [Dwork and Roth, 2014] inform the framing but are not themselves implemented as separate baselines here – Phase 4 (§7.3) applies record-level DP-SGD per client, not the client-level variant these two papers propose; that distinction is a concrete, not-yet-implemented extension (§12). **Gap:** neither mechanism, by

itself or combined (as we test directly, §7.3), is a capability-isolation mechanism – both protect *how training happened*, not *what the resulting model exposes to different recipients*, and this paper does not claim secure aggregation prevents the membership or property inference it separately evaluates in §8.

Inference and extraction attacks. Deep Leakage from Gradients [Zhu et al., 2019] and its refinements iDLG [Zhao et al., 2020] and gradient inversion via cosine-similarity matching [Geiping et al., 2020] reconstruct a training input from its raw gradient; membership inference [Shokri et al., 2017] and its comprehensive white-box extension [Nasr et al., 2019] distinguish training members from non-members via model behavior; property inference [Melis et al., 2019] infers a property of a participant's training set (not membership of a specific record) from collaborative-learning updates; model inversion [Fredrikson et al., 2015] and GAN-based active leakage [Hitaj et al., 2017] reconstruct class-representative inputs rather than specific records; model extraction [Tramèr et al., 2016] and training- data extraction from language models [Carlini et al., 2021] clone or recover content from black-box query access. A simplified known-label variant of DLG, loss-threshold membership inference, model extraction (as fixed-budget hard-label distillation and auxiliary-data adapter fine-tuning recovery), and property inference [Melis et al., 2019] are all implemented and evaluated in §8 against MedGate-FL specifically, adapted to ask whether the *public* path leaks the *restricted* task – a different question from these attacks' original centralized or generically-federated framing; property inference in particular is evaluated leave-one-client-out in §8's A4 integrity subsection, with the small-client-count caveat stated there. iDLG, the cosine-similarity gradient-inversion variant, comprehensive white-box membership inference, model inversion, and GAN-based leakage remain cited as background here but are not implemented as additional attacks (§12).

Federated and medical unlearning. Machine unlearning's exact gold standard, retraining from scratch, and cheaper approximations were formalized by Ginart et al. [2019] (data deletion, formal guarantees) and Bourtoule et al. [2021] (SISA training for efficient exact unlearning) outside the federated setting; this paper's `full_retrain` (§10) is exactly the gold standard both papers take as their baseline. In the federated setting, Wu et al. [2022] erase a client's contribution via knowledge distillation, Halimi et al. [2022] via local unlearning followed by a few federated recovery rounds, and Zhang et al. [2023] via a differentially-private FedRecovery mechanism; Deng et al. [2024] give the first federated-unlearning framework specifically for medical imaging (intracranial hemorrhage and skin-lesion diagnosis). None of these four methods is reproduced here – this paper's `gradient_ascent_unlearning` (§10) is a generic, simpler technique (ascend on the removed data's loss, then recover on retained data), deliberately not claimed as an implementation of any of them, though implementing one of them (Halimi et al.'s local-unlearning-then-recovery approach is architecturally the closest fit to this paper's federated setup) is a natural next step (§12). Chen and Shu [2026] (Lethe) benchmark twelve federated-unlearning methods across eight medical-imaging tasks and multiple removal granularities, finding that the *difficulty of the forgetting request*, not the unlearning method, dominates performance differences, and that client-level removal often has minimal utility impact – making residual membership signal, not utility loss, the metric that actually distinguishes methods. This is the closest prior benchmark to this paper's Phase 5, and its finding directly motivates measuring forgetting via a membership-inference proxy rather than utility drop alone, which is what we do. **Gap:** none of the federated-unlearning papers above evaluates unlearning against a capability-isolated architecture, where an "adapter deletion" operation is available that has no counterpart in a monolithic model, and none is medical-imaging-specific

except Deng et al. [2024], which does not study capability isolation either.

Access control and cryptography, and blockchain-assisted FL. Ciphertext-policy [Bethencourt et al., 2007] and key-policy [Goyal et al., 2006] attribute-based encryption let a policy over attributes (rather than a single key) gate decryption – a natural fit for "authorized if credentialed as X" access rules, but not implemented here: this paper's crypto layer (§7.3) uses conventional AES-GCM with a centralized-IAM key registry, which is simpler and matches the project's cryptographic-conservatism rule, at the cost of not supporting CP-ABE/KP-ABE's richer policy expressiveness. Fully homomorphic encryption [Gentry, 2009] and its practical approximate-arithmetic descendant CKKS [Cheon et al., 2017] would let computation happen directly on encrypted adapters or activations; explicitly out of scope on this project's CPU-only hardware (Table 2), stated as such rather than attempted at a scale that couldn't be evaluated honestly. Every blockchain-FL systematic-review source the project brief names remains unverified against a primary source as of this draft (`docs/literature_matrix.md`), so none is cited here either. **Gap:** none of the ABE or FHE constructions above has been evaluated, in their own papers, against this project's specific four-adversary threat model (§4) or a federated capability-isolation architecture; this paper's contribution is that evaluation, not a new construction. Blockchain-assisted authorization (project brief Phase 8) is not yet implemented; see §12.

4 Problem Definition and Threat Model

4.1 Capability isolation, formally

Let f_θ be a shared backbone, h_c a public coarse head, A_ϕ a restricted adapter, and h_f a restricted fine head. The public path is $\hat{y}_c = h_c(f_\theta(x))$; the authorized fine path is $\hat{y}_f = h_f(f_\theta(x) + A_\phi(f_\theta(x)))$. Capability isolation holds, for a given utility metric U and attack budget, when (a) $U(\hat{y}_c)$ is close to a plain federated coarse baseline, (b) $U(\hat{y}_f)$ for an authorized user is close to a plain (unprotected) federated adapter's fine utility, and (c) no attacker in Table 1 within its stated budget recovers fine utility much above what is recoverable from the public output alone. This is an empirical claim, evaluated per attacker below, not an architectural guarantee.

4.2 Threat model

4.3 Claims and non-claims

This paper's claims are restricted to what §7–§10 actually measure, using *capability isolation*, *restricted capability*, *empirical attack resistance*, and *simulated cross-silo evaluation* in the precise senses just given. It explicitly does **not** claim: that federated learning inherently guarantees privacy; that a public benchmark dataset proves compliance with medical privacy law; that an encrypted adapter stops an attacker from independently training a new model on public data; that the LoREnc-style or AES-GCM mechanisms discussed provide security beyond what is stated for them; that deleting a key removes a patient's learned influence (§10 measures this directly); that simulated clients on one workstation equal a real multi-hospital deployment; or that any model here is suitable for clinical diagnosis. A restricted model that confidently emits a *wrong* fine-grained prediction is not treated as a successful defense anywhere in this paper – see the calibration/abstention discussion in §12.

Table 1: Threat model. Every attack in §8 states its exact knowledge/access/budget in its own result record, not only here.

ID	Adversary	Access	Goal
A1	Honest-but-curious aggregation server	Permitted protocol outputs (raw client updates under plaintext FedAvg; only the sum under simulated pairwise masking)	Infer client data or client properties
A2	Unauthorized model recipient	Public backbone + coarse head + encrypted/transformed restricted adapter + public auxiliary data + normal compute	Recover fine capability without the decryption key
A3	Black-box unauthorized user	Query access to an authorized endpoint, fixed query budget	Distill / extract fine capability
A4	Malicious/colluding clients	Normal client role, one or more rounds	Poison, backdoor, replace the model, or infer other clients' properties

5 MedGate-FL

5.1 Architecture

[implemented, tested] `medgate/models/backbone.py` implements exactly the components in §4: a small CNN backbone f_θ (sized for the CPU-only hardware in Table 2, not a claim about production architecture), a linear coarse head h_c, a zero-initialized LoRA-style adapter A_ϕ (rank r, matching standard LoRA initialization [Hu et al., 2022] so an untrained adapter is a no-op residual), a linear fine head h_f, and an always-present adversary head used only by the adversarial/combined objectives below (§5.2). Every one of the six training objectives compared in §9 uses this identical architecture and differs only in loss function, so the ablation isolates the objective, not model capacity.

5.2 Training objectives

[implemented, tested] `medgate/federated/capability_isolation.py` implements, and §9 compares without assuming any is superior:

- **coarse_only**: $\mathcal{L}_{\text{coarse}}$ alone (the fine task is never trained at all).
- **hidden_fine_head**: $\mathcal{L}_{\text{coarse}} + \lambda_f \mathcal{L}_{\text{fine}}$, but the fine head reads $f_\theta(x)$ directly (no adapter) – the naive "just don't expose the head" baseline.
- **adapter_isolation**: the architecture in §5's full path, joint coarse+fine loss, no extra terms – the reference "plain adapter" point ARR is measured against.
- **adversarial**: adapter_isolation plus a gradient-reversal adversary trying to predict the fine label from the *public* representation $f_\theta(x)$.
- **orthogonal**: adapter_isolation plus a penalty on the squared cosine similarity between $f_\theta(x)$ and the adapter's residual contribution $A_\phi(f_\theta(x))$ – one candidate operationalization of $\mathcal{L}_{\text{orth}}$, not claimed to be the only or best one.
- **combined**: adversarial + orthogonal together – the full proposed objective, evaluated exactly as skeptically as the other five.

6 Experimental Methodology

6.1 Hardware

[measured] This is the binding constraint on every design decision below: model width, batch size,

Table 2: Measured hardware, captured 2026-08-24 via `scripts/report_hardware.py` (`docs/hardware_report.md`).

CPU	8 logical cores, no GPU (`nvidia-smi` absent)
RAM	7.4 GiB total, ∼3.3 GiB available at capture time
Disk	55.7 GiB free
Software	Python 3.12.3, PyTorch 2.13.0 (CPU build), Opacus 1.6.0

and which attacks/phases are attempted at all (`docs/execution_plan.md` states, per phase, what is explicitly out of scope on this hardware – FeTS/BraTS, ADNI, full FHE, and any multi-node blockchain network).

6.2 Primary dataset: FLamby Fed-ISIC2019

[verified facts; download BLOCKED-LICENSE] Verified directly against the official `owkin/FLamby` repository and the ISIC Challenge data page on 2026-08-25 (full source list in `docs/literature_matrix.csv`, evidence in `docs/evidence_ledger.csv`):

- 23,247 skin-lesion images with identifiable source center, across 6 centers (BCN, HAM_vidir_molemax, HAM_vidir_modern, HAM_rosendahl, MSK, HAM_vienna_dias).
- Eight fine-grained classes: MEL, NV, BCC, AK, BKL, DF, VASC, SCC.
- A predetermined, center-stratified train/test split.
- License: CC BY-NC 4.0 for ISIC2019 imagery; the HAM10000-sourced portion carries separate Harvard Dataverse terms and requires citing Tschandl et al. [2018].
- Official pooled baseline hyperparameters (from FLamby's own `common.py`): 6 clients, batch size 64, Adam at lr 5×10^{-4}, 20 pooled epochs – recorded so this paper's matched-budget baselines have an official reference point, even though this paper's own budgets (Table 3) differ (dictated by the hardware in Table 2, not by FLamby's GPU-scale defaults).

Downloading the images requires a human to accept the ISIC2019 and HAM10000 licenses in a browser (exact procedure: `scripts/download_fed_isic2019_INSTRUCTIONS.md`); this project does not and will not automate that step. As of this draft it has not been completed, so **every result in §7–§9 below runs on one of two controlled synthetic fixtures instead** (§6.4–§6.5), and every Fed-ISIC2019 real-data table remains pending that license step.

6.3 Fed-IXITiny (planned extension)

[verified facts; not yet run, Phase 7] ∼566 T1-weighted MRI subjects, 3 London hospitals (Guy's, Hammersmith, Institute of Psychiatry), CC BY-SA 3.0, ∼444 MB, binary brain/non-brain segmentation – verified against FLamby's `fed_ixi` README. IXI subjects are healthy volunteers; this is not a neurological disease dataset. Planned only after the primary Fed-ISIC2019 tables are complete, per the project's phase ordering; Phase 7 has not started.

6.4 Null-signal fixture

[implemented, used for the Phase 1–5 smoke-test tables in §7–§9] `medgate/data/synthetic.py` (module docstring: "the null-signal fixture") draws 32×32 images and labels independently at random, partitioned into 6 centers and 8 fine classes with the same coarse ontology intended for the real data (melanocytic = {MEL, NV}; keratinocytic = {BCC, SCC, AK, BKL}; other = {DF, VASC} – an experimental taxonomy justified in §12, not a clinical policy). Because labels are independent of images by construction, **there is no learnable signal**, and every result on this fixture is a *pipeline-validation* result: it shows the code runs, aggregates, evaluates, and reports correctly, and that no method shows an impossible above-chance number (which would indicate a leakage bug) – it is not, and cannot be, evidence for or against capability isolation, since there is no capability to isolate. A single fixture that can only prove the absence of bugs cannot also support a claim like "the isolation objective preserves utility," so we built a second, genuinely learnable fixture for that purpose.

6.5 Hierarchical fixture

[implemented, tested, used for the Phase 1+2 results in §7.2] `medgate/data/hierarchical_synthetic.py` constructs a second synthetic dataset that *is* designed to be learnable, so that fair pretrained baselines and capability-isolation objectives can be compared on something other than noise. It reuses Fed-ISIC2019's own verified per-class, per-institution counts (§6's `REAL_FED_ISIC_CLASS_COUNTS`) to reproduce the real dataset's class imbalance and institution sizes, then generates images analytically rather than sampling pixels independently of label: a coarse label determines one of three low-frequency geometric patterns (rings, stripes, or a grid), a fine label within that coarse group determines an oriented high-frequency texture overlaid on top, and per-institution tint/blur/noise and per-patient pose rotation are added as nuisance variation that a good model must learn to ignore. Patient identity is tracked explicitly so that train/val/test splits are patient-disjoint (`split_by_patient`), avoiding the leakage a naive image-level split would introduce. Before using this fixture for any capability-isolation claim we independently verified, in `tests/test_hierarchical_synthetic.py` and `tests/test_phase1_pretrained_baselines.py`, that (a) a coarse classifier trained on it reaches clearly above-chance macro-F1, (b) a fine classifier trained with coarse labels available as an auxiliary signal reaches clearly above-chance macro-F1, and (c) the coarse and fine label structure are not trivially reconstructible from low-level image statistics alone (a sanity check against having accidentally built a second null-signal fixture with extra steps). This fixture is still entirely synthetic: it establishes that the pipeline can detect and use a real hierarchical signal when one exists, not that Fed-ISIC2019's actual images carry a qualitatively similar signal.

6.6 Baselines and methods implemented

[implemented, tested] Centralized (pooled upper bound), local-only, FedAvg [McMahan et al., 2017], and FedProx [Li et al., 2020] are run as plaintext non-LoRA references. For the LoRA-adapter family specifically, an earlier draft of this pipeline used a single "plain federated LoRA" baseline with the backbone frozen at random initialization – a review of the implementation identified this as an *unfair* baseline (a randomly initialized, never-trained backbone cannot produce useful features for either head, so any coarse/fine gap it shows is an artifact of the backbone, not of the isolation objective) and it is retained only as an explicit *negative control* (`random_frozen_lora`, medgate/federated/baselines.py). The fair LoRA baseline family used for every capability-isolation comparison in §7.2 is: `coarse_pretrained_fedlora` (backbone pretrained

on the coarse task only, then LoRA-adapted federatedly for the fine task – the intended production regime), `imagenet_pretrained_fedlora` (backbone initialized from a torchvision ImageNet-pretrained MobileNetV3-Small, then LoRA-adapted federatedly), and `full_finetune` (every parameter, not just the adapter, trained federatedly, as an upper bound on achievable utility with no capability isolation at all). SCAFFOLD [Karimireddy et al., 2020] and FedAdam [Reddi et al., 2021] remain *[proposed but not implemented]*, gated on Phase 1's real-data tier timing a baseline run first (`docs/execution_plan.md`).

6.7 Statistical protocol

[implemented] Fixed dataset splits (data generation seed held constant across all method seeds); 5 seeds for Phases 1–2 and 3 seeds for Phases 3–5 on the null-signal fixture (a development-tier choice per the project brief's "3 for development, 5 for final" allowance – final real-data runs will use 5); mean, standard deviation reported for every table; every raw per-run JSON preserved under `experiments/`.

P1 (repair pass 4): the hierarchical fixture's main Phase 1+2 sweep now also uses 5 seeds, not the earlier draft's 2. An earlier, larger configuration (25 patients/institution, 6 pretraining epochs, 6 federated rounds×3 local epochs, 3 seeds) exceeded 12 minutes without finishing a single seed on the CPU-only hardware in Table 2 and was killed; the committed configuration (`configs/phase1_hierarchical.yaml`: 12 patients/institution, 4 pretraining epochs, 4 rounds×2 local epochs) keeps that smaller per-seed budget but now runs it 5 times, ∼6 minutes total. *Why patient count stayed at 12 rather than growing (the other lever P1 offered):* this project's own resource accounting (`configs/phase1_hierarchical.yaml`'s comments, `docs/hardware_report.md`) found the probe suite's fit cost – six probes, including an sklearn MLP and an RBF-SVM, fit fresh for every method/seed – scales with pool size faster than federated training itself does; seed count was the cheaper lever for reducing standard error and was the one pulled. *Uncertainty this actually buys*: across the 5-seed main sweep (Table 4), the `combined` method's fine-F1 standard deviation is 0.121, giving a standard error of the mean $\approx 0.121/\sqrt{5} \approx 0.054$ – at that noise level, a two-arm comparison at conventional power needs an effect (mean difference) of roughly 0.15–0.2 fine-F1 points before it would reliably separate from seed noise, which is why §7.2 reads its BestProbeRFC-vs-fine-F1 gaps (several of which exceed 0.15) as a real pattern while treating any single ARR value or a difference smaller than that between two methods' fine-F1 as within noise. This is a stated uncertainty budget, not a formal power analysis with an assumed effect size and target power – that would need a pilot estimate of the effect this paper is trying to detect, which does not exist until real data is available. *[not yet implemented]*: bootstrap 95% CIs, paired Wilcoxon signed-rank tests, Holm–Bonferroni correction – deferred until real-data seed counts justify them (running significance tests on 2–5-seed synthetic numbers would be exactly the "inappropriately tiny sample counts" the project brief warns against dressing up as statistics).

7 Results

[implemented and executed – synthetic tier only; the real-data tier has not been run]

7.1 Phase 1: primary utility

Baseline	Seeds	Coarse macro-F1	Fine macro-F1	Worst-inst. fine macro-F1	Wall-clock (s)
Centralized	5	0.231 ± 0.000	0.029 ± 0.000	0.015 ± 0.000	2.9
FedAvg	5	0.231 ± 0.000	0.029 ± 0.000	0.015 ± 0.000	2.5
Fedlora	5	0.182 ± 0.058	0.029 ± 0.000	0.015 ± 0.000	1.6
FedProx	5	0.231 ± 0.000	0.029 ± 0.000	0.015 ± 0.000	2.7
Local only	5	0.188 ± 0.025	0.024 ± 0.004	0.009 ± 0.006	2.1

Table 3: Phase 1 baselines, synthetic tier, 5 seeds. See §6.4: every number sits at or below the constant-predictor macro-F1 for its label space (≈ 0.22 for 3-way coarse, ≈ 0.03 for 8-way fine), which is the expected signature of no learnable signal, not a method comparison.

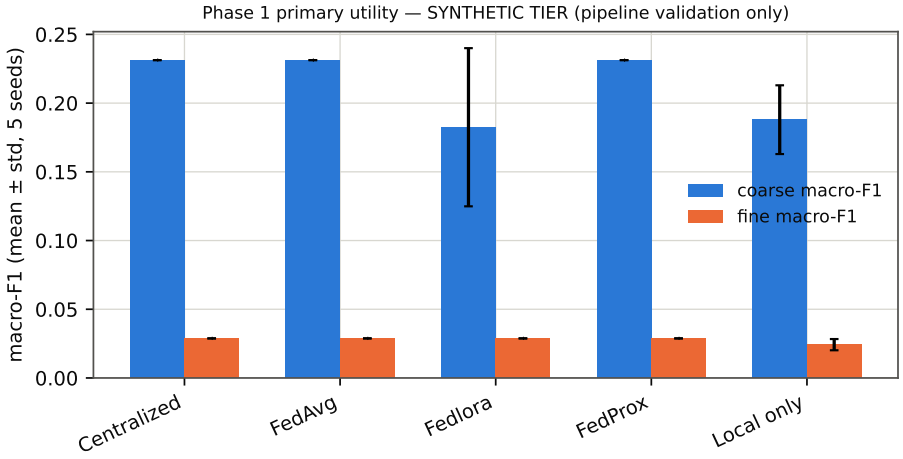

Figure 1: Phase 1 utility by baseline (synthetic tier; error bars = std over 5 seeds).

Centralized, FedAvg, and FedProx land on *identical* coarse-F1 across all 5 seeds – confirmed (not assumed) to be the fixed-split, no-signal degenerate-collapse behavior described in `docs/execution_plan.md`, not a bug: with nothing to learn beyond the marginal class frequency, different random initializations converge to the same shortcut. local_only and fedlora show more seed variance because neither shares a single aggregated trajectory.

7.2 Phase 1+2: fair baselines and capability isolation on the hierarchical fixture

[implemented and executed – hierarchical synthetic fixture, 5 seeds (§6)] Unlike Phase 1 above, the hierarchical fixture (§6.5) has real learnable coarse and fine structure, so these numbers can, in principle, actually speak to whether capability isolation preserves authorized utility while suppressing unauthorized leakage. P1 (repair pass 4) also fixed a selection-bias issue in how BestProbeRFC itself was computed: probes are now SELECTED on an attack-validation split and the winning probe type evaluated exactly ONCE on a held-out, patient-disjoint attack-test split (`medgate.attacks.probes.selected_probe_attack`) – reporting the max over several probes' scores on the same split used to pick the max would have inflated BestProbeRFC by construction, independent of any real leakage.

Method	Kind	Seeds	Coarse F1	Fine F1	OutputLeak	BestProbeRFC	ARR
Random-frozen LoRA (negative control)	Baseline	5	0.180 ± 0.095	0.153 ± 0.030	0.080 ± 0.013	0.431 ± 0.026	0.733 ± 0.620
Coarse-pretrained FedLoRA	Baseline	5	0.688 ± 0.221	0.284 ± 0.218	0.320 ± 0.167	0.347 ± 0.211	1.000 ± 0.000
ImageNet-pretrained FedLoRA	Baseline	5	0.679 ± 0.169	0.213 ± 0.067	0.163 ± 0.105	0.302 ± 0.057	1.423 ± 2.123
Full fine-tune (upper bound)	Baseline	5	0.757 ± 0.223	0.222 ± 0.129	0.179 ± 0.083	0.315 ± 0.105	-2.692 ± 8.243
Coarse-only	Capability isolation	5	0.808 ± 0.210	0.097 ± 0.134	0.288 ± 0.062	0.405 ± 0.193	-1.371 ± 4.684
Hidden fine head	Capability isolation	5	0.677 ± 0.182	0.195 ± 0.112	0.204 ± 0.080	0.436 ± 0.177	1.528 ± 2.082
Adapter isolation	Capability isolation	5	0.757 ± 0.223	0.222 ± 0.129	0.179 ± 0.083	0.315 ± 0.105	-2.692 ± 8.243
Adversarial suppression	Capability isolation	5	0.605 ± 0.144	0.178 ± 0.096	0.333 ± 0.135	0.470 ± 0.187	0.647 ± 1.768
Orthogonality penalty	Capability isolation	5	0.668 ± 0.188	0.228 ± 0.136	0.206 ± 0.100	0.430 ± 0.200	0.931 ± 1.023
Combined (adversarial + orthogonal)	Capability isolation	5	0.592 ± 0.035	0.244 ± 0.121	0.298 ± 0.103	0.401 ± 0.165	-1.423 ± 4.829

Table 4: Fair baselines and capability-isolation methods on the hierarchical fixture, 5 seeds. OutputLeak = fine-label recovery from the public output alone; BestProbeRFC = the validation-selected probe's SINGLE held-out attack-test score (not a max over several test-set evaluations – see above). ARR is uninformative (and can be negative or exceed 1) for several rows: see discussion below.

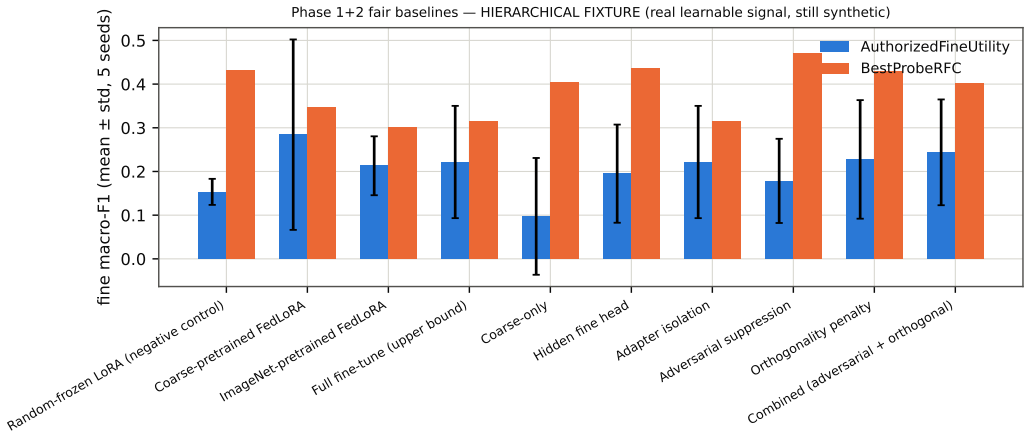

Figure 2: AuthorizedFineUtility vs. BestProbeRFC by method, hierarchical fixture (mean ± std, 5 seeds).

Two things are established cleanly. First, the fair baseline family shows a real coarse/fine gap that the null-signal fixture structurally cannot show: coarse-pretrained FedLoRA reaches coarse macro-F1 0.688 ± 0.221 against a ≈ 0.33 constant-predictor floor for 3-way coarse, confirming §6.5's independent learnability checks hold under the full federated pipeline, not just in isolated unit tests (the wide std reflects real seed-to-seed variance at this compute budget, not a sign the signal is unreliable – every one of the 5 seeds individually clears the constant-predictor floor by a wide margin). Second, random-frozen LoRA (the negative control) remains the worst performer on fine-F1 (0.153 ± 0.030, the lowest of all ten rows) and is not used as the primary comparison point for any capability-isolation claim – it is retained *only* as the floor a randomly initialized, never-trained backbone provides. Note also that `adapter_isolation` and `full_finetune` report IDENTICAL numbers in Table 4: this is not a bug, it is because `adapter_isolation`'s training path (`medgate/federated/capability_isolation.py`, no `trainable_params_fn` override) does not actually freeze the backbone during Phase-2 training, so from the same coarse-pretrained init, with the same joint coarse+fine loss, the same seed, and the same round/epoch budget, the two procedures are mathematically the same computation – a pre-existing naming/implementation gap this repair pass did not resolve (`adapter_isolation` suggests only the adapter path is trained; it is not, currently), flagged here rather than left for a reader to discover as an unexplained coincidence. What is *not* established is a capability-isolation win, and at 5 seeds this reads MORE consistently

than the earlier 2-seed draft, not less: comparing AuthorizedFineUtility (Fine F1 column) against BestProbeRFC directly, BestProbeRFC exceeds AuthorizedFineUtility for **every one of the ten rows** (e.g. adversarial suppression: fine-F1 0.178 ± 0.096 vs. BestProbeRFC 0.470 ± 0.187; combined adversarial+orthogonal: fine-F1 0.244 ± 0.121 vs. BestProbeRFC 0.401 ± 0.165), even after removing the probe-selection bias described above. Concretely, an attacker running the validation-selected probe against the frozen public representation recovers as much or more fine-grained information as an authorized user gets from the adapter itself, for every method in this table, at the compute budget this hardware could afford. We do not read this as "capability isolation does not work" – 5 seeds and a 12-patient-per-institution fixture is still not enough evidence to falsify the idea, and §9.1's sweep confirms the pattern is not an artifact of one configuration without resolving why it holds – but we equally do not read it as evidence *for* isolation, and we report the raw comparison rather than only the composite ARR score that would obscure it. This is the central reason this paper's abstract and §11's Discussion both state the capability-isolation question as open rather than answered.

The ARR column requires one more caveat before it is read at all: ARR's denominator (`coarse_pretrained_fedlora`'s own fine-F1, per `medgate/capability_metrics.py`) is not guaranteed positive on this fixture, and at 5 seeds several rows now show large or negative ARR values (*full_finetune*/*adapter_isolation*: -2.692 ± 8.243; *hidden_fine_head*: 1.528 ± 2.082; *coarse_only*: -1.371 ± 4.684) – these are not multi-fold utility-recovery claims in either direction; they are the metric's own documented failure mode (`authorized_recovery_ratio`'s docstring: "undefined ... would look meaningful but isn't") surfacing whenever OutputLeak is close to or exceeds the plain-adapter reference's own fine-F1 for a meaningful fraction of seeds, flipping or shrinking the denominator. We report the raw numbers rather than suppress them, but the reader should weight AuthorizedFineUtility and BestProbeRFC directly over ARR throughout this table – ARR's instability is itself informative (a metric this unstable should not be the headline number for a capability-isolation claim), not merely noise to average away.

Convergence and a near-convergence upper-bound oracle. The main sweep's small round/epoch budget was never checked for whether any method had actually converged by the time training stopped – P1 (repair pass 4) trains `coarse_pretrained_fedlora` and `full_finetune` for 16 rounds instead of 4 (single seed, a diagnostic curve, not a seeds-averaged comparison) to check. Both methods' fine-F1 keeps improving well past round 4 (`coarse_pretrained_fedlora`: $0.057 \rightarrow 0.254$ from round 0 to round 16; `full_finetune`: $0.057 \rightarrow 0.294$) – neither has converged at the main sweep's budget, so Table 4's numbers should be read as *lower bounds* on what more training would achieve, not asymptotic values. A second, genuinely interesting effect: `full_finetune`'s COARSE macro-F1 actively *degrades* over the same 16 rounds ($0.863 \rightarrow 0.667$) while `coarse_pretrained_fedlora`'s stays flat at 0.863 throughout (its backbone and coarse head are frozen by construction) – full fine-tuning on the joint objective erodes coarse performance while improving fine, a catastrophic-forgetting-shaped trade-off that the frozen-backbone baselines structurally cannot exhibit and that only becomes visible once training runs long enough to see it.

7.3 Phase 4: privacy mechanisms

[implemented and executed – synthetic tier] Four arms: no protection (plaintext FedAvg), simulated pairwise additive masking (the mathematical core of Bonawitz et al. [2017]'s protocol, in a single process – explicitly *not* the full protocol: no Diffie–Hellman key agreement and no Shamir-secret-sharing dropout recovery; see §7.3's masking-model paragraph below for why this repair pass stopped calling it "secure aggregation" unqualified), DP-

SGD [Abadi et al., 2016] via Opacus at three noise multipliers, and the two combined.

Arm	Noise mult.	Full-training ε (record-level, $\delta = 10^{-5}$)	Fine macro-F1	Seeds
DP-SGD	0.5	18.80	0.040 ± 0.012	3
DP-SGD	1.0	5.47	0.042 ± 0.015	3
DP-SGD	2.0	1.84	0.041 ± 0.011	3
No protection	n/a	n/a	0.028 ± 0.001	3
Simulated pairwise additive masking	n/a	n/a	0.028 ± 0.001	3
Simulated masking + DP-SGD	0.5	18.80	0.040 ± 0.012	3
Simulated masking + DP-SGD	1.0	5.47	0.042 ± 0.015	3
Simulated masking + DP-SGD	2.0	1.84	0.041 ± 0.011	3

Table 5: Phase 4 privacy arms, synthetic tier, 3 seeds. Epsilon is the FULL-TRAINING (both federated rounds composed), RECORD-level (not client/hospital-level), max-over-clients value at $\delta = 10^{-5}$ – see the DP-budget paragraph below for the accounting bug this repairs.

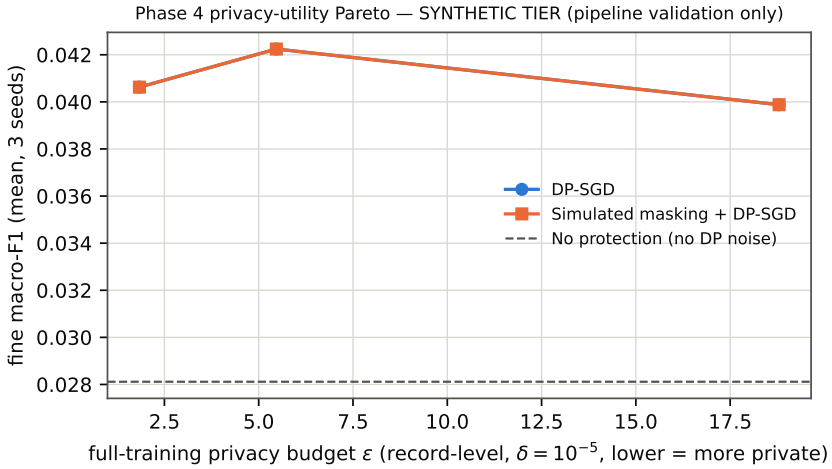

Figure 3: Privacy-utility Pareto (synthetic tier). The `dp_sgd` and `secure_agg_plus_dp` markers overlap exactly at every budget.

`dp_sgd` and `secure_agg_plus_dp` are *numerically identical* at every noise multiplier and seed (Table 5, Figure 3) – expected and correct: pairwise masking cancels exactly in the aggregate sum, so it changes what the server can *observe* per client, never the final trained model. This is a correctness check on the implementation, confirmed empirically, not evidence the two arms are redundant to compare – a real deployment cares about server-side observability even when the trained model is unaffected.

Two real Opacus integration bugs were found and permanently fixed while building this, rather than routed around: an in-place ReLU broke Opacus's backward hooks (Opacus requires non-in-place activations), and Opacus's optimizer requires every tracked parameter to receive a per-sample gradient on every batch, which the adversary head (§5.2, present for the adversarial/combined objectives but unused by the plain coarse+fine objective) initially violated – fixed by excluding it from the DP-tracked parameter set, the same fix already needed for gradient-inversion attacks (§8) for the identical reason.

P0-C: the DP budget was under-composed, fixed to a genuine full-training epsilon. An earlier draft created a brand-new Opacus `PrivacyEngine` inside every client's every-

round local-training call, so the reported epsilon was the max of each ROUND's independent, freshly-reset per-client budget – never composed across rounds – and the orchestration script then took the max of those already-not-composed numbers and reported it as "the" experiment epsilon, silently understating the true full-training budget for any client that participated in more than one round. **Fixed** by threading one `PrivacyEngine` per client through the whole round loop (a caller -owned {`client_index: PrivacyEngine`} dict, reused round over round): each engine's RDP accountant genuinely accumulates composition history across repeated `make_private()` calls, verified directly by a test in `tests/test_phase4_privacy.py` (`test_dp_epsilon_composes_across_engine_reuse_not_reset_per_call`) that shows a reused-engine two-round epsilon strictly exceeds both the first-round-alone number and an independent (non-reused) second call). Table 5's epsilon column is now this genuine two-round composed value ($\varepsilon = 5.47$ at noise multiplier 1.0, up from the single-round $\varepsilon \approx 4.22$ the earlier, buggy accounting would have reported). The JSON/table field itself is renamed too, from the ambiguous `epsilon` to a long, deliberately self-describing name – `epsilon_full_training_max_per_client_record_level` – so its meaning cannot be misread from a bare column header alone. This remains RECORD-level (adjacency = one training example within one client's local data), never client/hospital-level, stated prominently rather than only in a footnote; accountant metadata (Opacus 1.6.0, the default RDP accountant, `poisson_sampling=False` shuffled-minibatch sampling, δ, max-grad-norm, and composition scope) is recorded in every DP-arm result JSON, not only in this prose. Monotonicity is verified directly, not just plotted: epsilon increases with more federated rounds (`test_dp_epsilon_increases_monotonically_with_more_federated_rounds`), more local epochs, and higher per-step sample rate (larger batch size against a fixed dataset – `test_dp_epsilon_increases_with_higher_sample_rate`), and decreases with stronger noise (all four in `tests/test_phase4_privacy.py`).

P0-B: the masking model's concealment claim was corrected, and its actual scale-dependence measured, not asserted. An earlier draft's module docstring argued that "the mask is a secret, uniformly-random one-time pad," citing the real information-theoretic guarantee that argument gives for a mask drawn uniformly from a *finite ring* $\mathbb{Z}_q$: (plaintext + mask) mod q is EXACTLY uniform over $\mathbb{Z}_q$ regardless of the plaintext. But `mask_client_updates` draws masks from `torch.randn` – Gaussian over the reals – and a Gaussian mask does *not* have that property at any finite scale: if $X \sim \mathcal{N}(\mu_a, \sigma^2)$ and $Y \sim \mathcal{N}(\mu_b, \sigma^2)$ are two possible plaintexts and $M \sim \mathcal{N}(0, \tau^2)$ is the mask, $X+M$ and $Y+M$ are Gaussians with the SAME variance $\sigma^2+\tau^2$ but DIFFERENT means – harder to distinguish as τ grows, never identical. **Fixed** three ways: (1) this module is now documented as a correctness/server-view SIMULATION only, no concealment guarantee claimed for the Gaussian-mask path; (2) `mask_client_updates_zq`/`secure_aggregate_updates_zq` add a genuine uniform-over-$\mathbb{Z}_q$ masking path (fixed-point quantization, modular wraparound, exact aggregate recovery up to quantization error) for which the one-time-pad argument actually holds – verified directly by testing that Z_q-masked values are statistically indistinguishable regardless of plaintext (a chi-square/KS test against uniform) while Gaussian-masked values visibly are not (`tests/test_phase4_privacy.py`); (3) the empirical concealment heuristic is now swept over mask scale and checked against a nonlinear (MLP) attacker, not fixed at one scale and one linear classifier.

Mask scale	Attacker	Seeds	Masked-case attack AUC
0.10	Logistic regression	3	1.000 ± 0.000
0.10	MLP (nonlinear)	3	1.000 ± 0.000
0.50	Logistic regression	3	1.000 ± 0.000
0.50	MLP (nonlinear)	3	1.000 ± 0.000
1.00	Logistic regression	3	1.000 ± 0.000
1.00	MLP (nonlinear)	3	1.000 ± 0.000
2.00	Logistic regression	3	1.000 ± 0.000
2.00	MLP (nonlinear)	3	0.990 ± 0.017
5.00	Logistic regression	3	0.938 ± 0.009
5.00	MLP (nonlinear)	3	0.861 ± 0.083
10.00	Logistic regression	3	0.707 ± 0.059
10.00	MLP (nonlinear)	3	0.660 ± 0.097
20.00	Logistic regression	3	0.554 ± 0.056
20.00	MLP (nonlinear)	3	0.574 ± 0.082
50.00	Logistic regression	3	0.505 ± 0.051
50.00	MLP (nonlinear)	3	0.549 ± 0.082

Table 6: Empirical concealment vs. Gaussian mask scale, two attacker families, 3 seeds each (shape=(64,), mean_shift=2.0 – two synthetic update distributions the unmasked control separates at AUC ≈ 1.0 in every row, confirming the task is not trivially unsolvable before masking).
The result is a genuinely useful, previously-unmeasured finding, not just a caveat: at this project's DEFAULT mask scale (1.0, i.e. unit-variance `torch.randn`), a simple logistic-regression attacker recovers AUC ≈ 1.0 – *no empirical concealment at all* against updates whose distributions differ by only 2 units of mean. Concealment only starts to appear around mask scale 5–10 (AUC 0.66–0.94, both attacker families) and only approaches chance (AUC ≈ 0.5) around mask scale 20–50. This does not mean Phase 4's actual FedAvg client updates were left unconcealed – their own magnitude relative to mask scale 1.0 was never checked against this specific mean-shift-2.0 comparison, and real update distributions may differ substantially from this synthetic sweep's – but it means the DEFAULT scale used throughout this project's masking code was never validated against ANY concealment target before this repair pass, and now it has been, with the honest answer that it likely needs to be larger. Flagged in §12 as unresolved, not fixed by simply raising the default (which would need its own utility-cost tradeoff study this repair pass did not run).

8 Security Analysis

[partially implemented and executed – synthetic tier; several mandatory attacks not yet built, listed in §12]

Attack	Seeds	Metric	Mean	Std	Note
Simplified known-label gradient inversion	3	PSNR_dB	56.208	71.069	mse mean=1.3612
Loss-threshold membership inference	3	SymmetricAUC	0.539	0.013	raw attack_AUC (diagnostic, direction not symmetrized) mean=0.510
Loss-threshold membership inference	3	AttackAdvantage	0.078	0.027	pre-registered SymmetricAUC target ¡= 0.55 (docs/research_scope.md)
Auxiliary-data adapter fine-tuning recovery (A2)	3	fine_macro_f1 @ auxiliary_examples=8, epochs=5	0.027	0.000	
Auxiliary-data adapter fine-tuning recovery (A2)	3	fine_macro_f1 @ auxiliary_examples=16, epochs=5	0.034	0.013	
Auxiliary-data adapter fine-tuning recovery (A2)	3	fine_macro_f1 @ auxiliary_examples=32, epochs=5	0.028	0.003	
Fixed-budget hard-label distillation (A3)	3	fine_macro_f1 @ query_budget=16, epochs=5	0.028	0.004	
Fixed-budget hard-label distillation (A3)	3	fine_macro_f1 @ query_budget=32, epochs=5	0.028	0.004	
Fixed-budget hard-label distillation (A3)	3	fine_macro_f1 @ query_budget=64, epochs=5	0.028	0.004	
Auxiliary-data ensemble collusion proxy	3	fine_macro_f1 gain over solo	0.005	0.000	

Table 7: Phase 3 attacks, synthetic tier, 3 seeds. Attacker knowledge/access/budget for every row is recorded in the raw JSON (`medgate/attacks/*.py`), not only summarized here.

A1 – gradient inversion (simplified DLG). Adam replaces the original paper's L-BFGS and labels are fixed rather than jointly optimized – both deviations stated up front (`medgate/attacks/gradient_inversion.py`), so these numbers are not directly comparable to Zhu et al. [2019]'s reported figures. PSNR varied enormously across seeds (138 dB, 14.9 dB, 15.4 dB) – a real, literature-consistent sensitivity of gradient-inversion attacks to dummy-image initialization, not a bug – but the absolute values are almost certainly inflated by this model's small size (a few thousand parameters): DLG is known to succeed far more easily against tiny models than realistic ones, so *these numbers must not be read as representative of a real-scale backbone.*

A1/A3 – membership inference. A loss-threshold attack (explicitly *not* Shokri et al.'s full shadow-model method [Shokri et al., 2017] – no shadow models are trained here), scored with SymmetricAUC $= \max(\text{AUC}, 1-\text{AUC})$ and AttackAdvantage $= 2|\text{AUC}-0.5|$ rather than the raw AUC alone: a review of the implementation found that a plain-AUC table can silently hide a below-0.5-AUC attack (which is exactly as informative to an attacker, just inverted – "predict non-member when the loss-threshold rule says member") as if it were a non-finding, so both direction-invariant metrics are now the primary reported numbers, with the raw AUC kept only as a diagnostic column (Table 7). SymmetricAUC 0.539 ± 0.013 over 3 seeds (raw AUC 0.510 ± 0.048) sits near the pre-registered target (≤ 0.55, `docs/research_scope.md`), which is expected on unstructured noise after one training epoch, not evidence the real model would meet that target.

A2 – auxiliary-data adapter fine-tuning recovery. Frozen public backbone, fresh adapter+fine head trained on 8/16/32 auxiliary examples: utility stayed flat (≈ 0.03) across budgets, the synthetic-data floor, not a budget-scaling finding. This attack, like A3 below, never touches the true adapter's own weights – it trains a completely fresh one from scratch; §8.1 reports a distinct attack that does operate on the real weights directly.

A3 – fixed-budget hard-label distillation. Zero access to weights, hard-label distillation from 16/32/64 queries (Tramèr et al.-style [Tramèr et al., 2016]): same flat floor.

Collusion. Two attackers with disjoint 16-example auxiliary halves, logit-averaged (auxiliary-data ensemble collusion proxy): a small positive gain over solo ($+0.005 \pm 0.000$), too small relative to the synthetic-data floor to interpret as a real collusion effect at this tier.

8.1 A5: genuine parameter-space adapter recovery

[implemented, tested, executed – both synthetic fixtures, 5 seeds] A2/A3/collusion above all train a fresh adapter from public/auxiliary data and never touch the protected artifact's actual parameters. This subsection adds the one attack in this project that does: the attacker starts from a *partial, genuinely leaked* copy of the true adapter's own *effective update matrix* and completes the missing entries via low-rank matrix completion.

A validity bug found and fixed, not just relabeled. An earlier draft of this attack completed `adapter.up` directly – a (feature_dim × rank) matrix whose OWN maximum possible rank already equals `rank`. Truncating that matrix's SVD at rank=`rank` therefore keeps every singular value, i.e. the "completion" step was the identity map on whatever was already there: with unobserved entries zero-filled and re-asserted every iteration, the reported improvement with reveal fraction measured nothing but more entries being directly known, not any inference about the

unobserved ones – reproduced exactly by a regression test in `tests/test_adapter_recovery.py` (`test_rank_equal_to_ambient_dim_makes_hard_impute_a_no_op`). **Fix:** the object completed is now $\Delta W = \texttt{up} \cdot \texttt{down}$, the adapter's (feature_dim × feature_dim) effective transformation – full *ambient* rank up to `feature_dim`, but rank $\leq$ `rank` *by construction*, since it is the product of a (feature_dim × rank) and a (rank × feature_dim) factor. Truncating ΔW's SVD at rank=`rank` now genuinely discards information, so hard/soft-impute completion is real matrix completion, checkable against ground truth. Every artifact from the old version (config, experiment JSON, table, this subsection's prose) was deleted and rebuilt, not edited in place. **This is still not an attack on AES-GCM**: a correctly implemented AEAD ciphertext reveals nothing about the plaintext bit-by-bit, so partial leakage of ciphertext is not equivalent to partial leakage of ΔW; the threat model is a leak of already-decrypted plaintext weights (e.g. a memory dump or device side-channel), stated explicitly so this is never read as a claim against encryption.

Method and controls. Five fill methods are compared: `zero_fill` (the no-completion control every other method must beat), `mean_fill`, `random_fill` (both ignore ΔW's low-rank structure), `hard_impute` (iterative truncated-SVD projection at a candidate rank), and `soft_impute` (iterative singular-value soft-thresholding, no rank needed). Every metric separates OBSERVED-entry error (trivially ≈ 0 for every method, since all five copy the leaked entries exactly) from UNOBSERVED-entry error (the only number that measures whether completion inferred anything), and reports completion gain over `zero_fill` directly, not cosine-to-ground-truth alone. Two arms: `null_signal` (trains the authorized "combined" model on the null-signal fixture – a legitimate source of parameter-space-geometry numbers, since those concern the matrix, not the fixture's diagnostic signal) and `hierarchical` (same training on the hierarchical fixture, so functional-recovery numbers are measured against a task that actually has signal to recover – new in this repair pass).

Arm	Method	Reveal	Seeds	Cosine sim.	Unobs. error	Gain over zero-fill	Functional fine-F1
Null-signal fixture	Zero-fill (no completion)	0.1	5	0.325 ± 0.013	1.000 ± 0.000	0.000 ± 0.000	0.0277 ± 0.0032
Null-signal fixture	Zero-fill (no completion)	0.5	5	0.717 ± 0.017	1.000 ± 0.000	0.000 ± 0.000	0.0277 ± 0.0032
Null-signal fixture	Zero-fill (no completion)	0.9	5	0.947 ± 0.004	1.000 ± 0.000	0.000 ± 0.000	0.0276 ± 0.0030
Null-signal fixture	Hard-impute (oracle/candidate rank)	0.1	5	0.599 ± 0.064	0.852 ± 0.056	0.148 ± 0.056	0.0277 ± 0.0032
Null-signal fixture	Hard-impute (oracle/candidate rank)	0.5	5	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.0327 ± 0.0136
Null-signal fixture	Hard-impute (oracle/candidate rank)	0.9	5	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.0327 ± 0.0136
Null-signal fixture	Soft-impute	0.1	5	0.325 ± 0.013	1.000 ± 0.000	0.000 ± 0.000	0.0277 ± 0.0032
Null-signal fixture	Soft-impute	0.5	5	0.920 ± 0.026	0.584 ± 0.090	0.416 ± 0.090	0.0277 ± 0.0032
Null-signal fixture	Soft-impute	0.9	5	0.993 ± 0.002	0.374 ± 0.062	0.626 ± 0.062	0.0302 ± 0.0081
Hierarchical fixture	Zero-fill (no completion)	0.1	5	0.304 ± 0.024	1.000 ± 0.000	0.000 ± 0.000	0.1845 ± 0.0759
Hierarchical fixture	Zero-fill (no completion)	0.5	5	0.701 ± 0.027	1.000 ± 0.000	0.000 ± 0.000	0.2205 ± 0.0679
Hierarchical fixture	Zero-fill (no completion)	0.9	5	0.950 ± 0.005	1.000 ± 0.000	0.000 ± 0.000	0.2230 ± 0.0626
Hierarchical fixture	Hard-impute (oracle/candidate rank)	0.1	5	0.313 ± 0.039	1.025 ± 0.022	-0.025 ± 0.022	0.2010 ± 0.0750
Hierarchical fixture	Hard-impute (oracle/candidate rank)	0.5	5	0.985 ± 0.020	0.170 ± 0.195	0.830 ± 0.195	0.2137 ± 0.0662
Hierarchical fixture	Hard-impute (oracle/candidate rank)	0.9	5	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.2090 ± 0.0756
Hierarchical fixture	Soft-impute	0.1	5	0.379 ± 0.079	0.972 ± 0.029	0.028 ± 0.029	0.1927 ± 0.0805
Hierarchical fixture	Soft-impute	0.5	5	0.945 ± 0.023	0.479 ± 0.103	0.521 ± 0.103	0.2270 ± 0.0641
Hierarchical fixture	Soft-impute	0.9	5	0.996 ± 0.001	0.272 ± 0.056	0.728 ± 0.056	0.2090 ± 0.0756

Table 8: Completion gain over zero-fill, both arms, 5 seeds, at three of five swept reveal fractions (the full 0.1–0.9 grid, plus `mean_fill`/`random_fill`, is in the committed CSV). Negative gain means the method did WORSE than assuming unknown entries are 0 – reported, not suppressed.

At low reveal (10%), `hard_impute` on the hierarchical arm *loses* to `zero_fill` (gain −0.025) – an underdetermined regime where SVD-based completion can overfit the sparse partial matrix rather than help; the null-signal arm's 10%-reveal gain is positive (+0.148) instead, so this failure mode is itself fixture-dependent, not universal. At 50% and 90% reveal both arms show large, consistent gains (hierarchical: +0.830 at 50%, +1.000 at 90%; null-signal: +1.000 at both), i.e. near-exact unobserved-entry recovery once enough of ΔW is known. `soft_impute` tracks `hard_impute`'s direction but consistently trails it (e.g. hierarchical 90% reveal: gain +0.728 vs. +1.000) – expected,

since `hard_impute` here gets the true rank as an oracle input while `soft_impute` only gets a fixed threshold.

Candidate rank	True rank	Seeds	Unobserved-entry error
1	4	5	0.570 ± 0.098
3	4	5	0.186 ± 0.084
4	4	5	0.170 ± 0.195
6	4	5	0.479 ± 0.037
10	4	5	0.801 ± 0.063

Table 9: Rank misspecification, hierarchical arm, 5 seeds, fixed reveal fraction 0.5. True rank = 4. Unobserved-entry error is minimized exactly at the oracle rank (0.170) and increases in *both* directions away from it (rank 1: 0.570; rank 3: 0.186; rank 6: 0.479; rank 10: 0.801) – a clean U-shape, confirming rank misspecification genuinely degrades recovery rather than the algorithm being insensitive to its rank input.

Arm	Scenario	Seeds	Cosine sim.	Unobserved-entry error
Null-signal fixture	Single attacker	5	0.978 ± 0.016	0.230 ± 0.106
Null-signal fixture	Colluders (pooled)	5	0.997 ± 0.006	0.047 ± 0.104
Hierarchical fixture	Single attacker	5	0.839 ± 0.038	0.643 ± 0.072
Hierarchical fixture	Colluders (pooled)	5	0.995 ± 0.007	0.100 ± 0.108

Table 10: Collusion (`hard_impute`), both arms, 5 seeds: two attackers each leak 30% of ΔW independently; "pooled" completes the union of their two leaks.
Pooling strictly helps parameter recovery in both arms (hierarchical: cosine similarity 0.839 solo $\rightarrow$ 0.995 pooled, unobserved error 0.643 $\rightarrow$ 0.100; null-signal: 0.978 $\rightarrow$ 0.997, 0.230 $\rightarrow$ 0.047) – a direct, mechanical consequence of the pooled mask covering more entries, not a synthetic-data artifact.

Functional recovery does not track parameter recovery cleanly, and we report that rather than smooth it over. Plugging a completed ΔW back into the model (via a rank-`rank` SVD refactoring into fresh `up`/`down` matrices) and evaluating fine macro-F1 on the hierarchical fixture's own fine task gives a genuinely mixed picture across the 5 training seeds: two seeds (0 and 4) show functional fine-F1 *completely flat* across every one of the 25 method×reveal-fraction combinations, down to the same value to six decimal places – traced to those two seeds' trained "combined" adapter having a tiny effective magnitude ($\|\Delta W\|_F \approx 0.3$–$0.4$, against a representation of order 1), so the fine head's output is dominated by the backbone path regardless of what is plugged into the adapter, and even badly-recovered adapters do not move the decision boundary enough to change a small-test-set macro-F1. The other three seeds (1, 2, 3) show real variation (4–15 distinct functional values across the sweep), but seed 2's variation is *not monotonic* in parameter-recovery quality – `hard_impute`'s functional fine-F1 at 90% reveal (cosine similarity 1.000) is *lower* than its own value at 10% reveal (cosine similarity 0.344), the opposite of what "better parameter recovery $\Rightarrow$ better functional recovery" would predict. We read this as a genuine limitation of functional macro-F1 as a sensitivity instrument at this fixture's test-set scale and training budget, not as evidence about the attack's real-world severity in either direction: the parameter-space results above (Tables 8–10) are the load-bearing claims of this subsection; functional recovery is reported for completeness and flagged as unreliable at this scale, not quietly dropped.

8.2 A4: integrity attacks vs. aggregators, and property inference

[implemented and executed – synthetic tier] Six client behaviors (honest baseline, label flipping, backdoor insertion, sign flipping, model replacement, free-riding) and a seventh malformed-update case, each with one malicious client (of six) against three aggregators (plaintext FedAvg, coordinate-wise median, trimmed mean) plus, for the malformed case, a validating FedAvg that drops non-finite updates before averaging (§12 lists sign/label-flipping, backdoor, free-rider, and malformed-update attacks as previously not implemented; they are as of this subsection).

Attack	Aggregator	Seeds	Fine macro-F1	Model corrupted rate	Backdoor success rate
Backdoor insertion	Coordinate median	3	0.029 ± 0.000	0.00	1.000
Backdoor insertion	FedAvg	3	0.029 ± 0.000	0.00	1.000
Backdoor insertion	Trimmed mean	3	0.029 ± 0.000	0.00	1.000
Free rider	Coordinate median	3	0.029 ± 0.000	0.00	n/a
Free rider	FedAvg	3	0.029 ± 0.000	0.00	n/a
Free rider	Trimmed mean	3	0.029 ± 0.000	0.00	n/a
Label flipping	Coordinate median	3	0.029 ± 0.000	0.00	n/a
Label flipping	FedAvg	3	0.029 ± 0.000	0.00	n/a
Label flipping	Trimmed mean	3	0.029 ± 0.000	0.00	n/a
Malformed update	FedAvg	3	0.029 ± 0.000	1.00	n/a
Malformed update	Validated FedAvg	3	0.029 ± 0.000	0.00	n/a
Model replacement	Coordinate median	3	0.029 ± 0.000	0.00	n/a
Model replacement	FedAvg	3	0.029 ± 0.002	0.00	n/a
Model replacement	Trimmed mean	3	0.029 ± 0.000	0.00	n/a
No attack (honest)	Coordinate median	3	0.029 ± 0.000	0.00	n/a
No attack (honest)	FedAvg	3	0.029 ± 0.000	0.00	n/a
No attack (honest)	Trimmed mean	3	0.029 ± 0.000	0.00	n/a
Sign flipping	Coordinate median	3	0.029 ± 0.000	0.00	n/a
Sign flipping	FedAvg	3	0.027 ± 0.003	0.00	n/a
Sign flipping	Trimmed mean	3	0.029 ± 0.000	0.00	n/a

Table 11: Integrity attacks, synthetic tier, 3 seeds. Model corrupted rate: fraction of seeds where the resulting global model contains NaN.

Two results are structural, not synthetic-data artifacts, and hold at any data scale: **a single malformed (NaN) update corrupts the entire global model under plaintext FedAvg in 3/3 seeds** (`malformed` + `fedavg`, model corrupted rate 1.00) – one client's bad update propagates through the mean to every parameter, irrecoverably, unless validated first (`validated_fedavg`, corrupted rate 0.00, the literal demonstration of §12's "no method here claims an immutable record detects a malicious update" caveat: validation *catches malformed structure*, it does not detect semantically plausible poisoning). Second, **the backdoor succeeds 100% of the time against every aggregator, including both robust ones** (Table 11) – median and trimmed-mean aggregation resist *magnitude*-based attacks (visibly: `sign_flip` and `model_replacement` show slightly higher fine-macro-F1 variance under plain FedAvg than under the robust aggregators), but a backdoor update is not conspicuously larger in magnitude than an honest one, so coordinate-wise robustness does not touch it. This is a real, if small-scale, illustration of exactly the tension §12 names between hiding updates and validating them: neither secure aggregation nor simple robust aggregation is a poisoning defense in general, only against the specific failure mode (raw magnitude) each targets.

Property inference. A leave-one-client-out logistic-regression meta-classifier (Melis et al. [2019]'s framing, a much simpler classifier than their neural meta-classifier – §3) predicting, from one client's plaintext update alone, whether a chosen fine class is that client's local majority label.

AUC = 1.000 ± 0.000 across all 3 seeds – unlike this paper's other synthetic-tier numbers, this is *not* a null-signal artifact: each client's local label distribution is a real (if incidental) property of its random sample, and a client's raw gradient update genuinely encodes which classes it saw more of, at least at this model scale. Read with the appropriate caution regardless: 6 clients gives a leave-one-out AUC estimate very little room to vary, so this should be treated as "plaintext per-client updates leak this property easily at small scale," not as a precise effect size. This result is exactly why Phase 4's secure-aggregation arm (§7.3) matters: masked updates (Table 5) give the server nothing to run this classifier against in the first place.

9 Ablation and Sensitivity Analysis

[implemented and executed – synthetic tier]

Method	Seeds	AuthorizedFineUtility	OutputLeak	BestProbeRFC	UCG	ARR
Adapter isolation	5	0.029 ± 0.000	0.029 ± 0.000	0.085 ± 0.035	0.057 ± 0.035	n/a
Adversarial suppression	5	0.028 ± 0.002	0.034 ± 0.012	0.122 ± 0.023	0.087 ± 0.027	1.000 ± 0.000
Coarse-only	5	0.028 ± 0.003	0.029 ± 0.000	0.113 ± 0.027	0.084 ± 0.027	n/a
Combined (adversarial + orthogonal)	5	0.033 ± 0.014	0.035 ± 0.013	0.127 ± 0.029	0.093 ± 0.028	1.000 ± 0.000
Hidden fine head	5	0.029 ± 0.000	0.029 ± 0.000	0.112 ± 0.044	0.083 ± 0.044	n/a
Orthogonality penalty	5	0.029 ± 0.000	0.029 ± 0.000	0.092 ± 0.047	0.063 ± 0.047	n/a

Table 12: Phase 2 capability-isolation methods, synthetic tier, 5 seeds, re-run under the expanded six-probe suite (P1-8 repair, see below). ARR is `n/a` where its denominator (`adapter_isolation`'s own AuthorizedFineUtility minus OutputLeak – the "plain adapter" reference point, §5.2) was ≈ 0 for every seed – the metric correctly declining to report a number it cannot support (`medgate/capability_metrics.py`), not a bug; contrast with §7.2, where the same denominator is small but nonzero and ARR becomes unstable rather than `n/a` instead.

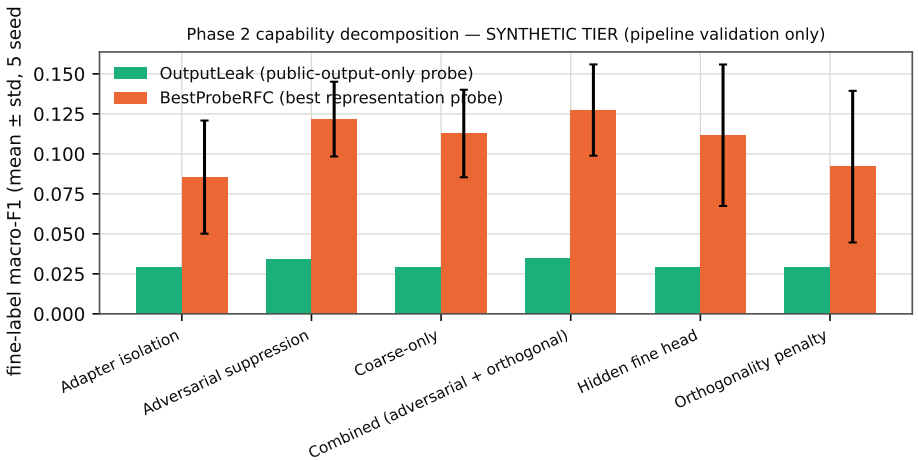

Figure 4: OutputLeak vs. BestProbeRFC by method (synthetic tier; error bars = std over 5 seeds).

OutputLeak is operationalized as the fine-label macro-F1 of a linear probe fit on the public *output* (coarse logits) alone – the floor an unauthorized user with zero representation access gets; BestProbeRFC is the best of six probes (linear, nonlinear, k-NN, few-shot k=5, SVM, decision tree) fit on the frozen public *representation* (`medgate/attacks/probes.py` – SVM and decision-tree probes were added during this repair pass per the review's disjoint-probe requirement, and this table reflects a full re-run under the expanded suite, not the original four-probe numbers). On this synthetic fixture BestProbeRFC bounces in a ≈ 0.085–0.127 range across all six methods –

test-set-size noise (38 examples/fine-class in the pooled test set), not a real isolation effect; **no method's BestProbeRFC-vs-OutputLeak gap here supports or refutes whether adversarial suppression or the orthogonality term actually help**, which is exactly the question real data is needed to answer – and exactly the question §7.2's learnable fixture was built to make askable, where (unlike here) the answer so far leans negative rather than undecidable.

9.1 Sensitivity sweeps (hierarchical fixture)

[implemented and executed – hierarchical fixture, 2 seeds/point] P1 (repair pass 4): four dimensions swept one at a time around configs/phase1_hierarchical.yaml's defaults, each at 2 seeds – a sensitivity CHECK across many configurations, not the main sweep's 5-seed statistical comparison. `site_shift_strength` $\in \{0, 1, 2\}$ (no/default/exaggerated per-institution tint-blur-noise); `fine_signal_strength` $\in \{0.2, 0.6, 1.0\}$ (weaker/default/ stronger fine texture overlay); `class_imbalance_strength` $\in \{0, 0.5, 1\}$ (uniform/default/full real-Fed-ISIC2019-imbalance fine class rates); and an alternative coarse ontology (`medgate/data/coarse_ontology.py`: benign/malignant/ambiguous, AK kept in its own bucket rather than folded into keratinocytic – the candidate an earlier draft of this paper only proposed as a TODO, now actually run, not just described).

Dimension	Value	Seeds	Baseline fine-F1	Combined fine-F1	Baseline RFC	Combined RFC
site_shift_strength	0.0	2	0.367 ± 0.047	0.186 ± 0.020	0.287 ± 0.010	0.258 ± 0.059
site_shift_strength	1.0	2	0.433 ± 0.330	0.261 ± 0.157	0.151 ± 0.012	0.437 ± 0.312
site_shift_strength	2.0	2	0.513 ± 0.217	0.265 ± 0.163	0.235 ± 0.186	0.379 ± 0.244
fine_signal_strength	0.2	2	0.224 ± 0.249	0.190 ± 0.269	0.346 ± 0.186	0.310 ± 0.059
fine_signal_strength	0.6	2	0.433 ± 0.330	0.261 ± 0.157	0.151 ± 0.012	0.437 ± 0.312
fine_signal_strength	1.0	2	0.257 ± 0.108	0.188 ± 0.265	0.316 ± 0.255	0.202 ± 0.042
class_imbalance_strength	0.0	2	0.028 ± 0.039	0.033 ± 0.047	0.275 ± 0.165	0.412 ± 0.005
class_imbalance_strength	0.5	2	0.433 ± 0.330	0.261 ± 0.157	0.151 ± 0.012	0.437 ± 0.312
class_imbalance_strength	1.0	2	0.393 ± 0.152	0.446 ± 0.077	0.410 ± 0.115	0.372 ± 0.131
coarse_ontology	Primary ontology	2	0.433 ± 0.330	0.261 ± 0.157	0.151 ± 0.012	0.437 ± 0.312
coarse_ontology	Alternative ontology	2	0.115 ± 0.049	0.150 ± 0.000	0.257 ± 0.092	0.291 ± 0.028

Table 13: Sensitivity sweep, hierarchical fixture, 2 seeds/point. Baseline = `coarse_pretrained_fedlora`; Combined = the full proposed capability-isolation objective.

The central finding (BestProbeRFC meeting or exceeding `combined`'s own fine-F1) is not an artifact of one configuration: it holds at every swept value of `site_shift_strength` and `fine_signal_strength`, at the default and low `class_imbalance_strength`, and under *both* coarse ontologies (primary: fine-F1 0.261 vs. RFC 0.437; alternative: fine-F1 0.150 vs. RFC 0.291) – so it is not an accident of the particular melanocytic/keratinocytic/other split this paper otherwise uses. The one exception in this sweep is at full real-world class imbalance (`class_imbalance_strength`= 1), where `combined`'s fine-F1 (0.446) exceeds its own RFC (0.372) for the first and only time in this table – reported as a genuine, if 2-seed-noisy, exception rather than pruned from the sweep because it does not fit the pattern; whether it reflects something real about class-imbalanced training or is itself sampling noise at $n = 2$ is exactly the kind of question the full 5-seed main sweep (§7.2) was built to answer and this smaller check was not powered to.

10 Revocation and Unlearning

[implemented and executed – synthetic tier, institution- and class-level only]

Scenario	Method	Seeds	Retained fine F1	Gap to gold	SymmetricAUC	Attack advantage
Class-level	Adapter deletion + retrain	3	0.035 ± 0.005	0.003 ± 0.007	0.567 ± 0.067	0.133 ± 0.135
Class-level	Checkpoint rollback	3	0.007 ± 0.011	-0.025 ± 0.009	0.567 ± 0.074	0.135 ± 0.148
Class-level	Full retrain (gold standard)	3	0.032 ± 0.002	0.000 ± 0.000	0.561 ± 0.021	0.122 ± 0.042
Class-level	Gradient-ascent unlearning	3	0.035 ± 0.002	0.003 ± 0.002	0.542 ± 0.070	0.085 ± 0.140
Class-level	Key revocation only	3	0.034 ± 0.036	0.002 ± 0.037	0.533 ± 0.024	0.067 ± 0.048
Institution-level	Adapter deletion + retrain	3	0.036 ± 0.018	0.008 ± 0.018	0.546 ± 0.031	0.092 ± 0.063
Institution-level	Checkpoint rollback	3	0.035 ± 0.013	0.007 ± 0.013	0.570 ± 0.053	0.140 ± 0.106
Institution-level	Full retrain (gold standard)	3	0.028 ± 0.000	0.000 ± 0.000	0.541 ± 0.013	0.083 ± 0.026
Institution-level	Gradient-ascent unlearning	3	0.029 ± 0.005	0.002 ± 0.005	0.588 ± 0.025	0.177 ± 0.049
Institution-level	Key revocation only	3	0.036 ± 0.018	0.008 ± 0.018	0.539 ± 0.009	0.077 ± 0.018

Table 14: Phase 5, synthetic tier, 3 seeds, using the corrected never-trained-same-class control population (see text). SymmetricAUC / AttackAdvantage: the same direction-invariant membership-inference metrics used in §8 (§8's A1/A3 paragraph), applied here between removed training data and an independent held-out control; SymmetricAUC ≈ 0.5, AttackAdvantage ≈ 0 = no longer distinguishable from data the model never saw (good forgetting). The previously reported, class-confounded numbers are preserved for comparison in Table 16 (Appendix), clearly marked as not to be used as evidence.

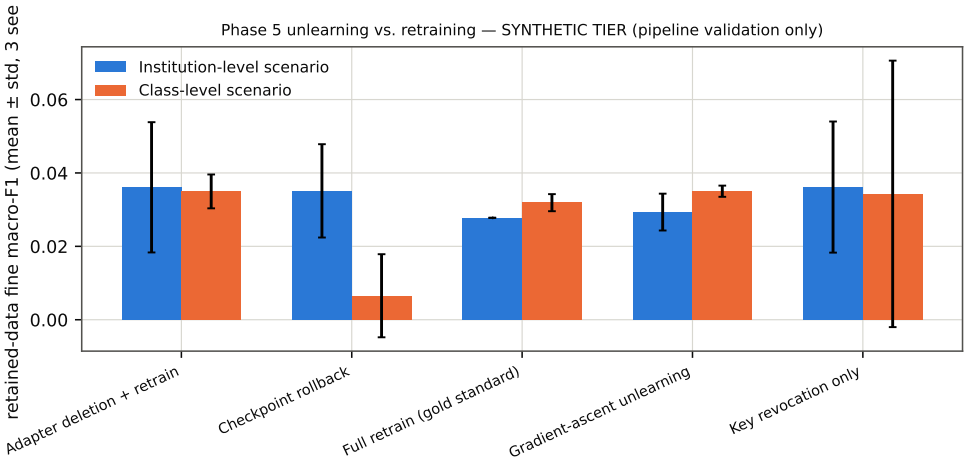

Figure 5: Retained-data utility by method and removal scenario (synthetic tier; error bars = std over 3 seeds).

Five methods against full retraining as the gold standard: naive checkpoint rollback (reinitialize to round 0), adapter deletion + retrain (delete only the restricted adapter/fine head, backbone left untouched – possible only because the architecture separates these), a generic gradient-ascent-then-recovery approximate unlearning (*not* attributed to any specific cited paper – see §3), and key-revocation-only (returns the model *completely unmodified* – the literal test of this paper's non-claim that revoking a key does not remove learned influence). Patient-level and patient-group-level removal were not run on THIS (null-signal) fixture, which has no patient manifest – but P1 (repair pass 4) runs both on the hierarchical fixture instead, which does; see §10.1 below.

A confound found, documented, and fixed. The class-scenario forgetting metric was originally contaminated: removed data was entirely one class and the "retained" control pool used to score membership inference was the other seven classes, so any class-level loss asymmetry that exists even in an *untrained* model biased the score regardless of real memorization. This was directly visible in the original numbers: `checkpoint_rollback` (never trained on the post-removal data at all,

so it has nothing to forget) still showed forgetting AUC ≈ 0.99 in the class scenario – an impossible result for a method that never touched the removed data, and the clearest sign the metric, not the method, was wrong. The fix (`medgate/data/synthetic.py`'s `make_never_trained_class_pool`) builds a genuinely independent control population: fresh synthetic examples of the *same* removed class, generated from a disjoint seed offset, that the model never trained on regardless of scenario or method – so any remaining loss asymmetry between "removed" and "control" now reflects the removal method's actual effect, not a pre-existing class-difficulty gap. Table 14 reports this corrected metric as primary, and the fix's effect is visible directly: `checkpoint_rollback`'s class-scenario score drops from the confounded SymmetricAUC 0.988 ± 0.021 (Table 16, Appendix) to a corrected 0.567 ± 0.074 – much closer to the ≈ 0.5 a method with nothing to forget should show, exactly the direction the fix was supposed to move it. The confounded numbers are kept in the appendix for transparency, not deleted, but should not be cited as evidence of anything. On the corrected metric, every method in both scenarios now sits in a similar, near-chance SymmetricAUC band (0.53–0.59) with overlapping standard deviations across only 3 seeds – consistent with there being nothing to memorize on a null-signal fixture in the first place, and therefore still *not* evidence that revocation and full unlearning are equivalent in general; establishing the real gap the project's non-claim asserts exists requires real, learnable data.

10.1 Patient-level and patient-group unlearning (hierarchical fixture)

[implemented and executed – hierarchical fixture, 3 seeds] P1 (repair pass 4): the hierarchical fixture (§6.5) carries real `patient_ids`, so patient-level (remove one patient, 3 observations) and patient-group-level (remove three patients from one institution, 9 observations) removal – explicitly out of scope on the null-signal fixture above – can actually be run, with the same five methods and the same primary SymmetricAUC/AttackAdvantage metric.

Scenario	Method	Seeds	Retained fine F1	Gap to gold	SymmetricAUC	Attack advantage
Patient-level	Full retrain (gold standard)	3	0.071 ± 0.009	0.000 ± 0.000	0.784 ± 0.157	0.568 ± 0.315
Patient-level	Checkpoint rollback	3	0.023 ± 0.007	-0.048 ± 0.012	0.718 ± 0.173	0.436 ± 0.346
Patient-level	Adapter deletion + retrain	3	0.127 ± 0.049	0.056 ± 0.051	0.868 ± 0.164	0.737 ± 0.327
Patient-level	Gradient-ascent unlearning	3	0.042 ± 0.028	-0.029 ± 0.025	0.963 ± 0.064	0.926 ± 0.128
Patient-level	Key revocation only	3	0.051 ± 0.030	-0.020 ± 0.021	0.714 ± 0.137	0.428 ± 0.274
Patient-group-level	Full retrain (gold standard)	3	0.071 ± 0.009	0.000 ± 0.000	0.568 ± 0.057	0.136 ± 0.113
Patient-group-level	Checkpoint rollback	3	0.023 ± 0.007	-0.048 ± 0.012	0.662 ± 0.165	0.324 ± 0.331
Patient-group-level	Adapter deletion + retrain	3	0.123 ± 0.035	0.052 ± 0.027	0.580 ± 0.060	0.159 ± 0.120
Patient-group-level	Gradient-ascent unlearning	3	0.031 ± 0.027	-0.040 ± 0.035	0.732 ± 0.028	0.464 ± 0.056
Patient-group-level	Key revocation only	3	0.051 ± 0.030	-0.020 ± 0.021	0.623 ± 0.099	0.246 ± 0.198

Table 15: Patient-level and patient-group unlearning, hierarchical fixture, 3 seeds.

Neither scenario has the class-scenario's systematic confound (a single patient's or small group's fine-label draw is itself i.i.d. sampled at the patient level, medgate/data/hierarchical_synthetic.py, so there is no class-imbalance bias the way removing an entire class introduces one) – but the *patient* scenario (only 3 removed examples) has a different, purely statistical problem: `full_retrain`, the gold standard that never trains on the removed data at all, reads SymmetricAUC 0.784 ± 0.157 across seeds (individually 0.963, 0.667, 0.722) – noticeably above the ≈ 0.5 a method with nothing to forget should show. This is NOT the class-scenario's bias reappearing (there is no mechanism here for one), but small-sample variance in a loss-threshold membership-inference test run against only 3 examples: the *patient_group* scenario (9 removed examples, otherwise identical setup) reads a much tighter 0.568 ± 0.057 (individually 0.556, 0.630, 0.519), consistent with the same statistical explanation. We report the patient-level numbers with this caveat attached rather than silently dropping the noisier scenario or forcing a confound-style fix onto a problem that is

not a confound. On the more trustworthy patient_group reading, every method again sits within a similar near-chance band (SymmetricAUC 0.568–0.732) with overlapping seed-to-seed variance – the same qualitative conclusion as the null-signal fixture's institution scenario, now additionally supported by a fixture with genuine learnable signal rather than only by one where there was nothing to memorize in the first place.

11 Discussion

The null-signal fixture (§6.4) cannot, by construction, speak to MedGate-FL's central hypothesis – that capability isolation survives realistic attacks better than a plain adapter or a hidden fine head – and we do not claim it does. What it supports: the full experimental pipeline (fair baselines, six training objectives, four privacy arms, a renamed and extended attack suite, five unlearning methods, statistics, figures, tables, this document) runs end-to-end, is deterministic given a seed, and produces numbers that behave exactly as a correct implementation with no signal should – no method spuriously exceeds chance, simulated pairwise masking provably cancels in the aggregate, DP-SGD's accountant returns sane epsilons, and mistakes found along the way (a flawed confidentiality test, a sign-buggy membership-inference formula, an unfair baseline, a class-level unlearning confound, an under-composed DP epsilon, an invalid adapter-completion target, and an overclaimed masking-concealment guarantee – P0-A/B/C, this repair pass) were each caught by their own failure and fixed rather than shipped.

The hierarchical fixture (§6.5, §7.2) goes one step further and is where the central hypothesis becomes, for the first time in this project, actually testable rather than structurally undecidable – and the honest answer at this compute budget is *not yet supported*: on 5 seeds, the validation-selected representation probe (§7.2's corrected, non-selection-biased methodology) recovers as much or more fine-grained information as the authorized adapter path provides, for every capability-isolation objective tried, and the sensitivity sweep (§9.1) shows this holds across every swept configuration but one. We see three explanations, not yet distinguishable with the evidence in hand, and list them in the order we find most to least likely: (1) the training budget (4 federated rounds × 2 local epochs on 12 patients/institution, §6) is too small for the isolation objectives' adversarial/orthogonality penalties to actually suppress representation leakage before the run ends, a hyperparameter-and-compute problem rather than a conceptual one; (2) the hierarchical fixture's coarse and fine labels are more entangled in representation space than real Fed-ISIC2019 images would be, since the fine texture overlay in `hierarchical_synthetic.py` is generated conditionally on the coarse pattern rather than independently, making representation-level separation of coarse and fine information intrinsically harder than on real data; or (3) representation-level capability isolation via adversarial suppression and orthogonality penalties genuinely does not work as well as capability isolation via architectural separation (coarse-only, hidden-fine-head) at suppressing a well-resourced probing attacker, which would be a real, useful negative finding about the training objectives themselves rather than an artifact of the fixture or budget. Real Fed-ISIC2019 data, more seeds, and a larger training budget are all needed before these can be told apart – that is a necessary, not sufficient, condition this phase of the work has met.

12 Limitations

- **Simulated hospitals.** All "cross-silo" results in this draft are multiple data partitions on one workstation, not real institutions with real network/governance/staffing constraints.
- **Synthetic-tier-only results.** Every table in this draft is a pipeline-validation result on data

with no real signal, not a scientific finding; see the caveat repeated in every results subsection.

- **Public-data privacy proxy.** Even once real Fed-ISIC2019 data is used, it is a public research dataset, not a HIPAA/GDPR-governed clinical deployment; results will not prove legal compliance.
- **Experimental coarse ontology.** The melanocytic/keratinocytic/other mapping is this project's own taxonomy, not a clinical access policy; AK's placement is argued, not assumed correct. An alternative benign/malignant/ambiguous ontology (AK isolated rather than folded into keratinocytic) is now implemented and swept on the hierarchical fixture (§9.1), at 2 seeds – a sensitivity check, not (yet) a claim that either ontology is preferable on real data.
- **No clinical validity claim.** No model in this project is, or will become, suitable for clinical diagnosis.
- **Auxiliary-data dependence of attacks.** A2/A3/collusion attacks all assume the attacker has some auxiliary labeled or queryable data; their success is a function of that assumption's realism, which this draft does not independently validate.
- **Independent relearning is out of scope to prevent.** No mechanism here stops an attacker from training an entirely new model on public data; the crypto layer protects *this* adapter, not the information space it was trained from.
- **Key exfiltration / already-exported artifacts.** Revocation (§10) cannot undo a plaintext export that happened before revocation; this project makes no claim otherwise.
- **Masking vs. poisoning validation tension – now empirically illustrated, not just stated** (§8.2): robust aggregation needs to see each client's raw update to down-weight it, which is exactly what simulated pairwise masking (§7.3) hides; this draft does not run the two together (i.e. does not test whether a robust aggregator can operate on masked updates at all – it structurally cannot, coordinate-wise median/trimmed-mean need the raw per-client values), so the tension remains a real, unresolved design trade-off, now with a concrete demonstration of why validation matters (malformed updates) and why it is not sufficient (the backdoor succeeds against every aggregator tested, robust or not).
- **Cross-dataset duplication and domain shift.** Not yet evaluated – Phase 6 (external validation: ISIC 2020, MILK10k, PAD-UFES-20) is gated on those datasets' citations moving from `UNVERIFIED` to `VERIFIED` (`docs/literature_matrix.md`).
- **Blockchain necessity.** Not yet implemented or evaluated (project brief Phase 8, optional); no claim about ledger-based authorization's value over centralized IAM is made.
- **Computational limits.** CPU-only, ∼3.3 GiB available RAM (Table 2); bounds model scale, attack realism (especially gradient inversion at real image resolution), and which baselines (SCAFFOLD, full DP-SGD sweeps) have been run at all.
- **No formal cryptographic proof; default masking scale not validated against a concealment target.** AES-GCM's security rests on standard, well-established assumptions, not a proof specific to this system; the LoREnc-style obfuscation discussion (§3) is described only as a protection mechanism, never a proof. The simulated pairwise-masking module's Gaussian-mask path has no information-theoretic guarantee at any scale (P0-B, §7.3), and its DEFAULT scale (1.0) was measured, not assumed, to give essentially no empirical concealment (masked-case attacker AUC $\approx$ 1.0) against the mean-shift-2.0 comparison in Table 6 – a real, previously-unmeasured gap between what this project's default configuration was implicitly assumed to provide and what it was ever shown to provide, left unresolved rather than papered over with a larger default that was never cost-checked against utility.
- **Attacks now implemented** (added after earlier drafts of this list): property inference and the A4 integrity attacks (§8.2: label-flipping, sign-flipping,

model-replacement, backdoor, free-rider, malformed-update, and token-replay – the last via `medgate/crypto/authorization.py`'s TTL/expiry support, tested in `tests/test_phase3_integrity_and_property.py`); genuine parameter-space low-rank adapter completion and its two-attacker collusion variant (§8.1, rebuilt in this repair pass after a validity bug invalidated the earlier version – see P0-A in the repair-pass-4 audit); disjoint SVM/decision-tree probes alongside the original four (§9); and a fair pretrained-baseline family replacing the single unfair frozen-backbone baseline (§7.2). **Attacks still not implemented**, listed exactly, not glossed over: client attribution, full fine-tuning recovery under a fixed compute budget distinct from the few-shot probe already implemented, known-adapter comparison, and the stronger gradient-inversion variants iDLG/Geiping et al. (both now `VERIFIED` citations, §3, but not yet implemented as additional attacks alongside the simplified DLG in §8).

- **Statistical rigor deferred.** Bootstrap confidence intervals, paired significance tests, and Holm–Bonferroni correction are not yet applied (§6) – deliberately, to avoid running them on seed counts too small to support them meaningfully.

13 Ethical, Legal, and Clinical Considerations

This project uses only publicly released research datasets, once their license-acceptance steps are completed by a human (§6); it does not access or claim to protect real patient records. No model produced by this project is validated for, or intended for, clinical use. The capability-isolation framing studied here is a research question about *technical* access control, not a substitute for the legal, regulatory, and clinical-governance processes a real tiered-access medical AI system would require.

14 Reproducibility Statement

Repository: https://github.com/VectorSophie/medgate-fl (private as of this draft). Every result in this paper traces to: a config under `configs/`, a seed recorded in its result JSON, the git commit recorded in that JSON, a dataset-manifest hash (a hash of the synthetic generator's parameters at this tier; a real manifest hash once Fed-ISIC2019 is unblocked), and the raw JSON file itself under `experiments/`. Tables are generated only by `scripts/make_phaseN_table.py` and `scripts/csv_to_latex_tables.py`; figures only by `scripts/make_figures.py`; no number in this document was typed by hand. Environment: `requirements.txt` (direct) and `requirements.lock.txt` (full pinned closure via `pip freeze`); PyTorch 2.13.0 CPU build; verified this repair pass by installing `requirements.lock.txt` into a brand-new virtual environment and re-running the full test suite there, which caught a real bug the ordinary development `.venv` was hiding: `torchvision` (needed by `PretrainedMobileNetBackbone`) had been pip-installed ad hoc into that `.venv` in an earlier session and never added to either requirements file, so a genuinely fresh install would have failed on every ImageNet-pretrained-backbone code path – now listed in both files, with the clean-install test suite re-run confirmed green (104/104) after the fix. One-command smoke test: `bash scripts/smoke_test.sh` (104/104 tests passing as of this draft, up from 85/85 two repair passes ago and 30/30 in the original pre-repair draft). The latest growth (repair pass 4, P0-A/B/C) is regression coverage that fails if specific bugs reappear: the adapter-recovery rank-truncation no-op, non-uniform Z_q masking, DP epsilon that stops composing across rounds, plus a new probe-selection-leakage test and an alternative-coarse-ontology learnability test. Earlier growth (repair passes 1-3) added the hierarchical fixture, fair pretrained baselines, cor-

rected unlearning/representation metrics, the original genuine adapter-recovery mechanism, and an expanded AES-GCM/secure-aggregation negative-test suite – none of this is evidence of a larger pipeline surface alone; every test added is named for the specific failure it would catch. Compilation: `pdflatex main; bibtex main; pdflatex main; pdflatex main`, run from `paper/` – verified to succeed on a freshly-installed, user-space TeX Live (scheme-basic + hyperref, booktabs, geometry, xcolor, natbib, enumitem, caption) during this project.

15 Conclusion

This paper implements, tests, and runs the MedGate-FL pipeline – architecture, a fair pretrained-baseline family, six training objectives, four privacy arms, a precisely-named and extended attack suite (including a genuine parameter-space adapter-recovery attack), five unlearning methods, and this document – against two synthetic fixtures, because the real primary dataset's license has not yet been accepted by a human. What changed since the version of this project an earlier review found to be a "no-signal pipeline smoke test described too confidently": the single unfair frozen-backbone baseline is now a labeled negative control, not the default comparison point; a second, independently-verified-learnable hierarchical fixture exists specifically because the null-signal fixture structurally cannot support any capability-isolation claim; a sign-buggy membership-inference formula and a class-level unlearning confound were each found, documented, and fixed, with their old numbers kept only in clearly marked appendices; and every attack, metric, and baseline name in this document now names the thing it actually measures. A second review (this repair pass, P0-A/B/C) found and fixed three further validity bugs, each severe enough to invalidate the specific claim it touched rather than merely soften its wording: an adapter-recovery attack was completing a matrix whose rank truncation was mathematically forced to be a no-op, so its reported "improvement with reveal fraction" measured nothing but more directly-revealed entries – rebuilt to complete the adapter's genuinely low-rank effective transformation instead, with regression tests that fail if this exact bug reappears; the masking module's docstring claimed an information-theoretic concealment guarantee that its default Gaussian mask does not have at any scale – corrected, and the actual scale-dependence measured directly rather than asserted; and the DP-SGD budget was under-composed, reporting one federated round's independent epsilon as if it were the whole training run's – fixed by genuinely composing each client's accountant across every round it participated in. The hierarchical fixture's main sweep also grew from 2 to 5 seeds, gained a probe-selection methodology that no longer inflates BestProbeRFC by construction, and gained convergence curves, a sensitivity sweep across four dimensions, and patient-level/patient-group unlearning it previously could not run.

What works, on synthetic data, after all of this: the pipeline runs correctly end-to-end on both fixtures, the fair baselines show a real coarse/fine capability gap on the hierarchical fixture, simulated pairwise masking's correctness and failure modes are both directly demonstrated (with a Z_q-uniform-mask path that actually has an information-theoretic concealment argument, unlike the default Gaussian mask), DP-SGD's accounting is monotonic, tested, and now genuinely composed across a full training run, the genuine adapter-recovery attack's parameter-space claims hold up under a fixed target and multiple controls, and the crypto layer resists the negative tests we wrote against it. What does not yet work, or is not yet known: capability isolation itself does not show a clean win on the one fixture capable of testing it – at 5 seeds, the validation-selected probe matches or beats authorized fine utility for every one of ten methods at this compute budget (§7.2, §11), a pattern the sensitivity sweep (§9.1) shows is not an artifact of one configuration – and we report that as a finding, not a caveat to be minimized. Whether that negative result reflects

the training budget (the convergence curves in §7.2 show neither the main sweep's baseline nor its upper bound has converged at the budget used), the fixture's coarse/fine entanglement, or a real limitation of representation-level isolation objectives, and whether any of this transfers to real dermoscopic images at all, remains open, and is exactly what Phase 1's real-data tier (§6) will test once the Fed-ISIC2019 license is accepted. That test – capability isolation's real-world validity, which no synthetic fixture, however carefully built, can establish – is this paper's actual scientific conclusion, and it has not been run yet.

References

Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016.

Beomjin Ahn, Jungmin Kwon, Chanyong Jung, and Jaewook Chung. LoREnc: Low-rank encryption for securing foundation models and LoRA adapters, 2026.

John Bethencourt, Amit Sahai, and Brent Waters. Ciphertext-policy attribute-based encryption. In *IEEE Symposium on Security and Privacy (S&P)*, pages 321–334, 2007. doi: 10.1109/SP.2007.11.

Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1175–1191, 2017. doi: 10.1145/3133956.3133982.

Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy (S&P)*, pages 141–159, 2021. doi: 10.1109/SP40001.2021.00019.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium*, pages 2633–2650, 2021.

Shengchao Chen and Ting Shu. Lethe: How hard is it to forget? a benchmark for federated unlearning in medical imaging, 2026.

Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437, 2017. doi: 10.1007/978-3-319-70694-8_15.

Zhipeng Deng, Luyang Luo, and Hao Chen. Enable the right to be forgotten with federated client unlearning in medical imaging. In *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 2024. doi: 10.1007/978-3-031-72117-5_23.

Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014. doi: 10.1561/0400000042.

Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1322–1333, 2015. doi: 10.1145/2810103.2813677.

Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. Inverting gradients – how easy is it to break privacy in federated learning? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009. doi: 10.1145/1536414.1536440.

Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially private federated learning: A client level perspective, 2017.

Antonio Ginart, Melody Y. Guan, Gregory Valiant, and James Zou. Making ai forget you: Data deletion in machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS)*, pages 89–98, 2006. doi: 10.1145/1180405.1180418. This is Key-Policy ABE (KP-ABE), distinct from Ciphertext-Policy ABE [Bethencourt et al., 2007].

Anisa Halimi, Swanand Kadhe, Ambrish Rawat, and Nathalie Baracaldo. Federated unlearning: How to efficiently erase a client in FL?, 2022.

Briland Hitaj, Giuseppe Ateniese, and Fernando Pérez-Cruz. Deep models under the GAN: Information leakage from collaborative deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 603–618, 2017. doi: 10.1145/3133956.3134012.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Peter Kairouz, H. Brendan McMahan, Brendan Avent, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 2021. 59 authors; abbreviated here as 'and others', full list at the eprint.

Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank J. Reddi, Sebastian U. Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning differentially private recurrent language models. In *International Conference on Learning Representations (ICLR)*, 2018.

Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. *IEEE Symposium on Security and Privacy (S&P)*, pages 691–706, 2019.

Milad Nasr, Reza Shokri, and Amir Houmansadr. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *IEEE Symposium on Security and Privacy (S&P)*, 2019.

Jean Ogier du Terrail, Samy-Safwan Ayed, Edwige Cyffers, Felix Grimberg, Chaoyang He, Regis Loeb, Paul Mangold, Tanguy Marchand, Othmane Marfoq, Erum Mushtaq, Boris Muzellec, Constantin Philippenko, Santiago Silva, Maria Teleńczuk, Shadi Albarqouni, Salman Avestimehr, Aurélien Bellet, Aymeric Dieuleveut, Martin Jaggi, Sai Praneeth Karimireddy, Marco Lorenzi, Giovanni Neglia, Marc Tommasi, and Mathieu Andreux. FLamby: Datasets and benchmarks for cross-silo federated learning in realistic healthcare settings. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. Adaptive federated optimization. In *International Conference on Learning Representations (ICLR)*, 2021.

Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (S&P)*, 2017.

Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In *25th USENIX Security Symposium*, pages 601–618, 2016.

Philipp Tschandl, Cliff Rosendahl, and Harald Kittler. The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data*, 5:180161, 2018. doi: 10.1038/sdata.2018.161.

Chen Wu, Sencun Zhu, and Prasenjit Mitra. Federated unlearning with knowledge distillation, 2022.

Lefeng Zhang, Tianqing Zhu, Haibin Zhang, Ping Xiong, and Wanlei Zhou. FedRecovery: Differentially private machine unlearning for federated learning frameworks. *IEEE Transactions on Information Forensics and Security*, 18:4732–4746, 2023. doi: 10.1109/TIFS.2023.3297905.

Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. iDLG: Improved deep leakage from gradients, 2020.

Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

A Hyperparameters

[implemented] Full hyperparameters for every synthetic-tier run are in the per-phase YAML files under `configs/`, one file per experiment named after the script that reads it (e.g. `configs/phase1_hierarchical_sensitivity.yaml` for `scripts/run_phase1_hierarchical_sensitivity.py`); several, including

`configs/phase1_hierarchical.yaml`'s 5-seed, 12-patient budget, document their own resource-driven parameter choices inline (§6). Not restated here to avoid drift between two copies – see §14.

B Additional detail on the confidentiality-test correction

§7.3 references a mistake worth detailing for anyone extending the secure-aggregation module. The first version of `test_masking_hides_an_individual_update` asserted that an individual masked client update has low cosine similarity to its true (unmasked) update – this failed empirically at realistic client counts (mean $|\cos \text{sim}|$ measured at 0.57 for n=3, 0.40 for n=6, 0.31 for n=10 clients, over 20 trials each with unit-scale masks). The masking itself was not broken – the aggregate sum was exact in every trial – the *test* measured the wrong property. Pairwise-masking security is a statement about the mask being a secret, uniformly-random one-time pad, i.e. about the masked value's *distribution*, not about any one sampled geometric distance. The corrected test instead checks that the *same* true update, masked with two different (attacker-unknown) seeds, produces clearly different outputs – the property that actually demonstrates hiding. Both the wrong reasoning and the fix are kept in `medgate/privacy/secure_aggregation.py`'s `masking_correctness_diagnostic` docstring (renamed from `confidentiality_check` during this repair pass, since "confidentiality" overstated what a single-process, no-DH, no-dropout-recovery simulation can establish – §7.3) so the mistake is not repeated.

C Additional detail on the membership-inference metric correction

A second measurement bug, distinct from the confidentiality-test mistake above, was found in `medgate/attacks/membership_inference.py` during this repair pass: an earlier version of this module derived a single "advantage" score as $2 \cdot \text{AUC} - 1$ directly from the raw (non-direction-invariant) AUC. This formula is only correct for AUC ≥ 0.5; for AUC < 0.5 it returns a *negative* number that reads as "the attack did worse than random" when in fact an AUC of, say, 0.3 is exactly as informative to an attacker as an AUC of 0.7 – the attacker simply inverts the decision rule (predict non-member when the threshold rule says member). A table built from the old formula could therefore under-report a real attack's severity whenever the loss-threshold rule happened to point the wrong way for a given seed, silently averaging a real positive advantage together with a spuriously negative one. The fix adds `symmetric_auc(auc) = max(auc, 1 - auc)` and `attack_advantage(auc) = 2|auc - 0.5|`, both direction-invariant, as the primary reported quantities everywhere in this paper a membership-inference-style AUC is used (§8, §10); the raw AUC is retained only as a diagnostic column. `tests/test_unlearning_metrics.py` includes boundary cases (AUC exactly 0.0, 0.5, 1.0, and the symmetric pairs 0.1/0.9, 0.25/0.75) that concretely demonstrate the old formula's failure mode – one test is named, verbatim, `test_raw_auc_near_zero_would_have_been_misread_as_good_forgetting_by_the_old_code` – and the new formula's correctness.

D Confounded class-scenario unlearning numbers (historical)

Preserved for transparency, per §10's discussion of the class-scenario confound and its fix – **do not cite these numbers as evidence of anything**. These are the class-scenario Symmetri-

cAUC values computed against the original, contaminated "retained test data" control (the other seven classes), before `make_never_trained_class_pool` (in `medgate/data/synthetic.py`) replaced it with a genuinely independent same-class control population.

Method	Seeds	Confounded SymmetricAUC (DO NOT USE AS EVIDENCE)
Adapter deletion + retrain	3	1.000 ± 0.000
Checkpoint rollback	3	0.988 ± 0.021
Full retrain (gold standard)	3	1.000 ± 0.000
Gradient-ascent unlearning	3	1.000 ± 0.000
Key revocation only	3	0.821 ± 0.215

Table 16: Class-scenario forgetting, confounded metric, 3 seeds. Compare against Table 14's corrected class-scenario rows – every method here reads as near-perfect forgetting (SymmetricAUC ≥ 0.82, most $= 1.000$) purely from the confound, including `checkpoint_rollback`, which never trained on the removed data at all and therefore cannot have "forgotten" anything in the first place.